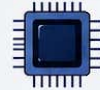
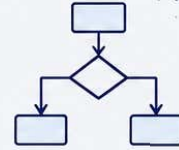


1010101010  
0101010101  
1010101010  
0101010101



# Infographics



## Visual Learning

Better understanding through visual content.



## Simplified Concepts

Complex topics explained in simple and visual form.



## Quick Revision

Easy to revise important points and diagrams.

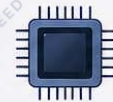
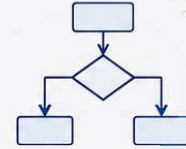


## Exam Ready

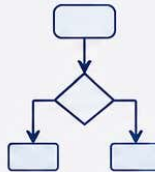
Helps in faster revision and better exam preparation.



1010101010  
0101010101  
1010101010  
0101010101



10110  
01001  
10101



# Unit-I



# BINARY ARITHMETIC

Number Systems • Conversions • Arithmetic Operations • Binary Codes.

"Bits are small but they do big things!"

## 1. NUMBER SYSTEMS

| System      | Base (Radix) | Digits       | Example           |
|-------------|--------------|--------------|-------------------|
| Binary      | 2            | 0, 1         | 1011 <sub>2</sub> |
| Octal       | 8            | 0 - 7        | 725 <sub>8</sub>  |
| Decimal     | 10           | 0 - 9        | 125 <sub>10</sub> |
| Hexadecimal | 16           | 0 - 9, A - F | 3A7 <sub>16</sub> |

- Binary: Used in computers (2 digits)
- Octal: Short form of binary (3 bits = 1 octal digit)
- Hexadecimal: Short form of binary (4 bits = 1 hex digit)

## 2. CONVERSION BETWEEN NUMBER SYSTEMS

### (A) Binary to Decimal

• Decimal value =  $\sum (\text{bit} \times 2^{\text{position}})$

Example: 101110<sub>2</sub>

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$= 16 + 0 + 4 + 2 + 0$$

$$= 22_{10}$$

### (B) Decimal to Binary

• Repeatedly divide by 2 and write remainders.

Example: 25<sub>10</sub>

|   |    |   |   |
|---|----|---|---|
| 2 | 25 | → | 1 |
| 2 | 12 | → | 0 |
| 2 | 6  | → | 0 |
| 2 | 3  | → | 1 |
| 2 | 1  | → | 1 |
|   | 0  |   |   |

Read  
bottom to top

25<sub>10</sub> = 11001<sub>2</sub>

### (C) Binary to Octal

• Group binary digits in sets of 3 from right.

Example: 110101<sub>2</sub>

|     |     |
|-----|-----|
| 110 | 101 |
| 6   | 5   |

= 65<sub>8</sub>

### (D) Octal to Binary

• Convert each octal digit to 3-bit binary.

Example: 725<sub>8</sub>

|     |     |     |
|-----|-----|-----|
| 7   | 2   | 5   |
| 111 | 010 | 101 |

= 111010101<sub>2</sub>

### (E) Binary to Hexadecimal

• Group binary digits in sets of 4 from right.

Example: 110111010<sub>2</sub>

|      |       |
|------|-------|
| 1101 | 11010 |
| D    | D     |

= DD<sub>16</sub>

### (F) Hexadecimal to Binary

• Convert each hex digit to 4-bit binary.

Example: 3A7<sub>16</sub>

|      |      |      |
|------|------|------|
| 3    | A    | 7    |
| 0011 | 1010 | 0111 |

= 001110100111<sub>2</sub>

### Quick Reference

- 1 hexadecimal digit = 4 binary bits
- 1 octal digit = 3 binary bits
- Useful for data representation and simplification

"Same value, different representations!"



## 3. BINARY ADDITION

Rules:

- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 0 = 1
- 1 + 1 = 10 (carry 1)

Example:

$$\begin{array}{r} 1\ 1\ 0\ 1 \\ +\ 1\ 0\ 1\ 1 \\ \hline 1\ 1\ 0\ 0\ 0 \end{array}$$

Same as decimal addition, but base-2.

## 4. BINARY SUBTRACTION

• Use 1's complement or 2's complement method.

• Example (2's complement method): Subtract 1011<sub>2</sub> from 1101<sub>2</sub>

1 2's complement of 1011  
→ 0101

2 Add with 1101

$$\begin{array}{r} 1101 \\ +\ 0101 \\ \hline 1\ 1\ 0\ 1\ 0\ 1 \end{array}$$

(carry ignored)

Result = 0010<sub>2</sub>

## 5. BINARY MULTIPLICATION

• Similar to decimal multiplication.

• Example: 1101 × 101

$$\begin{array}{r} 1\ 1\ 0\ 1 \\ \times\ 1\ 0\ 1 \\ \hline 1\ 1\ 0\ 1\ (\times 1) \\ 0\ 0\ 0\ 0\ (\times 0) \\ 1\ 1\ 1\ 0\ 1\ (\times 1) \\ \hline 1\ 0\ 0\ 0\ 1 \end{array}$$

Partial products are added.

## 6. BINARY DIVISION

• Similar to decimal long division.

• Example: 1101<sub>2</sub> ÷ 11<sub>2</sub>

$$\begin{array}{r} 11 \overline{) 1101} \quad 11 \\ - 11 \\ \hline 0011 \\ - 11 \\ \hline 00 \end{array}$$

Quotient = 11<sub>2</sub>  
Remainder = 00<sub>2</sub>

## 7. BINARY CODES

### (a) 8421 BCD (Binary Coded Decimal)

- Each decimal digit is represented by 4 bits.
- Example: 59<sub>10</sub> = 0101 1001 (8421 BCD)

### (b) Excess-3 Code

- Add 3 to each decimal digit, then convert to binary.
- Example: 5 → 5 + 3 = 8 → 1000
- So, 59 → 1000 1100 (Excess-3)

### (c) Gray Code

- Successive numbers differ by only one bit.

| Decimal   | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   |
|-----------|-----|-----|-----|-----|-----|-----|-----|-----|
| Gray Code | 000 | 001 | 011 | 010 | 110 | 111 | 101 | 100 |

- Used in error reduction, encoders, position sensors.



"Different Number Systems... Same Value, Different Forms!"



# Binary Number System and their Interconversion

(Binary  $\rightleftharpoons$  Decimal  $\rightleftharpoons$  Octal  $\rightleftharpoons$  Hexadecimal)

"Numbers change form but never lose their value!"

## 1. Number System

- A number system is a way to represent numbers using a set of symbols (digits).
- It has a base (radix) and a set of digits.
- In digital electronics, we mainly use Binary (base-2).

## 2. Types of Number Systems

| System      | Base (Radix) | Digits Used  | Example           |
|-------------|--------------|--------------|-------------------|
| Binary      | 2            | 0, 1         | 1011 <sub>2</sub> |
| Octal       | 8            | 0 - 7        | 725 <sub>8</sub>  |
| Decimal     | 10           | 0 - 9        | 125 <sub>10</sub> |
| Hexadecimal | 16           | 0 - 9, A - F | 3AF <sub>16</sub> |

## 3. Binary Number System

- Binary is a base-2 number system.
- It uses only two digits: 0 and 1.
- Each position has a weight of 2<sup>n</sup>, where n = 0, 1, 2, ... from right to left.

Example: (10110<sub>2</sub>)

$$\begin{array}{ccccccc} 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 0 & 1 & 1 & 0 \end{array}$$

$$= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ = 16 + 0 + 4 + 2 + 0 \\ = 22_{10}$$

- MSB (leftmost)  $\rightarrow$  highest weight
- LSB (rightmost)  $\rightarrow$  lowest weight

## 4. Binary to Decimal Conversion

Steps:

- Write the binary number.
- Assign weights 2<sup>n</sup> from right to left.
- Multiply each bit with its weight.
- Add all the values.

Example 1: 10110<sub>2</sub> to decimal

$$\text{Weights: } 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$$

$$\text{Bits: } 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1$$

$$= 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 \\ = 32 + 0 + 8 + 4 + 0 + 1 \\ = 45_{10}$$

## 8. Binary to Hexadecimal Conversion

Steps:

- Group binary digits in sets of 4 from right side.
- Convert each group to its hexadecimal equivalent.

Example 5: 1101110101<sub>2</sub> to hexadecimal

$$\begin{array}{ccc} 1101 & 1110 & 1011 \\ \downarrow & \downarrow & \downarrow \\ D & E & B \end{array}$$

$$1101110101_2 = DEB_{16}$$

If the leftmost group has less than 4 bits, add leading zeros.

## 12. Hexadecimal to Decimal Conversion

Steps:

- Multiply each digit with 16<sup>n</sup> (from right to left).
- Add all the values.

## 5. Decimal to Binary Conversion

Steps:

- Repeatedly divide the decimal number by 2.
- Write the remainder (0 or 1).
- Read the remainders from bottom to top.

Example 2: 25<sub>10</sub> to binary

$$\begin{array}{r|l} 2 & 25 \rightarrow 1 \\ 2 & 12 \rightarrow 0 \\ 2 & 6 \rightarrow 0 \\ 2 & 3 \rightarrow 0 \\ 2 & 1 \rightarrow 1 \\ & 0 \end{array} \quad \begin{array}{c} \uparrow \\ \text{Read} \\ \text{bottom} \\ \text{to top} \end{array}$$

$$25_{10} = 11001_2$$

## 9. Hexadecimal to Binary Conversion

Steps:

- Convert each hexadecimal digit to its 4-bit binary equivalent.
- Write all groups together.

Example 6: 3AF<sub>16</sub> to binary

$$\begin{array}{ccc} 3 & A & F \\ \downarrow & \downarrow & \downarrow \\ 0011 & 1010 & 1111 \end{array}$$

$$3AF_{16} = 001110101111_2$$

Example 9: 3AF<sub>16</sub> to decimal

$$\begin{array}{ccc} 16^2 & 16^1 & 16^0 \\ \downarrow & \downarrow & \downarrow \\ 3 & A_{(10)} & F_{(15)} \\ = 3 \times 256 + 10 \times 16 + 15 \times 1 \\ = 768 + 160 + 15 = 943_{10} \end{array}$$

## 6. Binary to Octal Conversion

Steps:

- Group binary digits in sets of 3 from right side.
- Convert each group to its octal equivalent.

Example 3: 1101011<sub>2</sub> to octal

$$\begin{array}{ccc} 001 & 101 & 011 \\ \downarrow & \downarrow & \downarrow \\ 1 & 5 & 3 \end{array}$$

$$1101011_2 = 153_8$$

If the leftmost group has less than 3 bits, add leading zeros.

## 10. Decimal to Octal Conversion

Steps:

- Repeatedly divide the decimal number by 8.
- Write the remainder (0 - 7).
- Read the remainders from bottom to top.

Example 7: 125<sub>10</sub> to octal

$$\begin{array}{r|l} 8 & 125 \rightarrow 5 \\ 8 & 15 \rightarrow 7 \\ 8 & 1 \rightarrow 1 \\ & 0 \end{array} \quad \begin{array}{c} \uparrow \\ \text{Read} \\ \text{bottom} \\ \text{to top} \end{array}$$

$$125_{10} = 175_8$$

## 7. Octal to Binary Conversion

Steps:

- Convert each octal digit to its 3-bit binary equivalent.
- Write all groups together.

Example 4: 725<sub>8</sub> to binary

$$\begin{array}{ccc} 7 & 2 & 5 \\ \downarrow & \downarrow & \downarrow \\ 111 & 010 & 101 \end{array}$$

$$725_8 = 111010101_2$$

## 11. Octal to Decimal Conversion

Steps:

- Multiply each digit with 8<sup>n</sup> (from right to left).
- Add all the values.

Example 8: 725<sub>8</sub> to decimal

$$\begin{array}{ccc} 8^2 & 8^1 & 8^0 \\ \downarrow & \downarrow & \downarrow \\ 7 & 2 & 5 \\ = 7 \times 64 + 2 \times 8 + 5 \times 1 \\ = 448 + 16 + 5 \\ = 469_{10} \end{array}$$

## 13. Quick Reference Table

| Decimal | Binary | Octal | Hexadecimal | Base | Dec to 8 | Dec to 16 | Summ |
|---------|--------|-------|-------------|------|----------|-----------|------|
| 0       | 0000   | 0     | 0           | 4    | 0100     | 4         | 4    |
| 1       | 0001   | 1     | 1           | 5    | 0101     | 5         | 5    |
| 2       | 0010   | 2     | 2           | 6    | 0110     | 6         | 6    |
| 3       | 0011   | 3     | 3           | 7    | 0111     | 7         | 7    |
| 4       | 0100   | 4     | 4           | 8    | 1000     | 10        | 8    |
| 5       | 0101   | 5     | 5           | 9    | 1001     | 11        | 9    |
| 6       | 0110   | 6     | 6           | 10   | 1010     | 12        | A    |
| 7       | 0111   | 7     | 7           | 11   | 1011     | 13        | B    |

"Practice Conversions Build a Stronger Logic!"

Convert  
Calculate  
Check  
Be Confident!

Different Representations... Same Value!

# OCTAL NUMBER SYSTEM AND THEIR INTERCONVERSION

Base-8 System • Conversions • Examples • Easy Steps

"Numbers are just in a different base!"

## 1. Octal Number System

- Octal number system is a base-8 number system.
- It uses 8 digits: 0, 1, 2, 3, 4, 5, 6, 7.
- Each position (digit) has a weight of  $8^n$ , where  $n = 0, 1, 2, \dots$  from right to left.
- It is a shorthand representation of binary data (since 1 octal digit = 3 binary bits).
- Used in digital electronics and computer systems.

Base = 8  
Digits = 0 to 7

## 2. Place Value in Octal

| Position (from right) | 4     | 3     | 2     | 1     | 0     |
|-----------------------|-------|-------|-------|-------|-------|
| Weight                | $8^4$ | $8^3$ | $8^2$ | $8^1$ | $8^0$ |
|                       | 4096  | 512   | 64    | 8     | 1     |

**Example:**

Find the decimal value of  $(725.4)_8$

**Solution:**

$$\begin{aligned}(725.4)_8 &= 7 \times 8^2 + 2 \times 8^1 + 5 \times 8^0 + 4 \times 8^{-1} \\ &= 7 \times 64 + 2 \times 8 + 5 \times 1 + 4 \times \frac{1}{8} \\ &= 448 + 16 + 5 + 0.5 \\ &= 469.5_{10}\end{aligned}$$

## 3. Octal to Decimal Conversion

**Steps:**

- Write the octal number.
- Multiply each digit with  $8^n$ .
- Add all the values.

**Example 1:**  $(347)_8$  to decimal

$$\begin{aligned}(347)_8 &= 3 \times 8^2 + 4 \times 8^1 + 7 \times 8^0 \\ &= 3 \times 64 + 4 \times 8 + 7 \times 1 \\ &= 192 + 32 + 7 \\ &= 231_{10}\end{aligned}$$

**Example 2:**  $(25.6)_8$  to decimal

$$\begin{aligned}(25.6)_8 &= 2 \times 8^1 + 5 \times 8^0 + 6 \times 8^{-1} \\ &= 16 + 5 + 6/8 \\ &= 21.75_{10}\end{aligned}$$

## 4. Decimal to Octal Conversion

**Steps:**

- Repeatedly divide the decimal number by 8.
- Write the remainders (0-7).
- Read the remainders from bottom to top.

**Example:** 100 to octal

$$\begin{array}{r|l} 8 & 100 \rightarrow \text{remainder } 4 \\ 8 & 12 \rightarrow \text{remainder } 4 \\ 8 & 1 \rightarrow \text{remainder } 1 \\ & 0 \end{array}$$

Read bottom to top

$$100_{10} = 144_8$$

## 5. Octal to Binary Conversion

**Steps:**

- Convert each octal digit to its 3-bit binary equivalent.
- Write all the 3-bit groups together.

**Example:**  $(725)_8$  to binary

$$\begin{array}{ccc} 7 & 2 & 5 \\ \downarrow & \downarrow & \downarrow \\ 111 & 010 & 101 \end{array}$$

$$(725)_8 = 111010101_2$$

**Example:**  $(3.6)_8$  to binary

$$\begin{array}{ccc} 3 & . & 6 \\ \downarrow & & \downarrow \\ 011 & . & 110 \end{array}$$

$$(3.6)_8 = 011.110_2$$

## 6. Binary to Octal Conversion

**Steps:**

- Group binary digits into sets of 3 bits from right side (and left side if needed).
- Convert each 3-bit group to its octal equivalent.

**Example:**  $11010110_2$  to octal

$$\begin{array}{ccc} 011 & 010 & 110 \\ \downarrow & \downarrow & \downarrow \\ 3 & 2 & 6 \end{array}$$

$$11010110_2 = (326)_8$$

**Example:**  $101.011_2$  to octal

$$\begin{array}{ccc} 101 & . & 011 \\ \downarrow & & \downarrow \\ 5 & . & 3 \end{array}$$

$$101.011_2 = (5.3)_8$$

## 7. Octal to Hexadecimal Conversion

**Steps:**

- Convert octal to binary (each digit  $\rightarrow$  3 bits).
- Group binary digits into 4 bits from right.
- Convert each 4-bit group to hexadecimal.

**Example:**  $(725)_8$  to hexadecimal

$$\begin{array}{ccc} 7 & 2 & 5 \\ \downarrow & \downarrow & \downarrow \\ 111 & 010 & 101 \\ \hline 0001 & 1101 & 0101 \\ \downarrow & \downarrow & \downarrow \\ 1 & D & 5 \end{array}$$

$$(725)_8 = (1D5)_{16}$$

## 8. Hexadecimal to Octal Conversion

**Steps:**

- Convert hexadecimal to binary (each digit  $\rightarrow$  4 bits).
- Group binary digits into 3 bits from right.
- Convert each 3-bit group to octal.

**Example:**  $(2F)_{16}$  to octal

$$\begin{array}{ccc} 2 & & F \\ \downarrow & & \downarrow \\ 0010 & 1111 & \\ \hline 000 & 101 & 111 \\ \downarrow & \downarrow & \downarrow \\ 0 & 5 & 7 \end{array}$$

$$(2F)_{16} = (57)_8$$

Practice More  
✓ Convert  
✓ Calculate  
✓ Be Confident  
✓ Keep Learning!

Same number, different base, same value!

Different Representation... Same Value! 😊

# HEXADECIMAL NUMBER SYSTEM AND THEIR INTERCONVERSION

## (Base-16 Number System)

Hexadecimal makes big numbers easy! 😊

### 1. Hexadecimal Number System

- Hexadecimal number system is a base-16 number system.
- It uses 16 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.
- Each position (digit) has a weight of  $16^n$ , where  $n = 0, 1, 2, \dots$  from right to left.
- It is a shorthand representation of binary data (since 1 hex digit = 4 binary bits).
- Widely used in computers, memory addressing, and digital electronics.

**Base = 16**  
**Digits = 0 - 9, A - F**

### 2. Hexadecimal Digits and Place Value

| Hex Digit     | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  |
|---------------|---|---|----|----|----|----|----|----|
| Decimal Value | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  |
| Hex Digit     | 8 | 9 | A  | B  | C  | D  | E  | F  |
| Decimal Value | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |

**Place Value (Right to Left):**

| Position | 4      | 3      | 2      | 1      | 0      |
|----------|--------|--------|--------|--------|--------|
| Weight   | $16^4$ | $16^3$ | $16^2$ | $16^1$ | $16^0$ |
|          | 65536  | 4096   | 256    | 16     | 1      |

### 3. Hexadecimal to Decimal Conversion

**Steps:**

- Write the hexadecimal number.
- Multiply each digit with  $16^n$ .
- Add all the values.

**Example 1:**  $(2AF)_{16}$  to decimal

$$\begin{aligned}(2AF)_{16} &= 2 \times 16^2 + A \times 16^1 + F \times 16^0 \\ &= 2 \times 256 + 10 \times 16 + 15 \times 1 \\ &= 512 + 160 + 15 \\ &= 687_{10}\end{aligned}$$

**Example 2:**  $(7C3)_{16}$  to decimal

$$\begin{aligned}(7C3)_{16} &= 7 \times 16^2 + C \times 16^1 + 3 \times 16^0 \\ &= 7 \times 256 + 12 \times 16 + 3 \\ &= 1792 + 192 + 3 \\ &= 1987_{10}\end{aligned}$$

### 4. Decimal to Hexadecimal Conversion

**Steps:**

- Repeatedly divide the decimal number by 16.
- Write the remainders (0-15).
- Read the remainders from bottom to top.

**Example:**  $945_{10}$  to hexadecimal

$$\begin{array}{r|l} 16 & 945 \rightarrow \text{remainder 1 (1)} \\ 16 & 59 \rightarrow \text{remainder 11 (B)} \\ 16 & 3 \rightarrow \text{remainder 3 (3)} \\ & 0 \end{array}$$

Read bottom to top

$$945_{10} = 3B1_{16}$$

Remainder value:  
10 = A, 11 = B,  
12 = C, 13 = D,  
14 = E, 15 = F

### 5. Hexadecimal to Binary Conversion

**Steps:**

- Replace each hex digit with its 4-bit binary equivalent.
- Write all the 4-bit groups together.

**Example:**  $(2AF)_{16}$  to binary

$$\begin{array}{ccc} 2 & A & F \\ \downarrow & \downarrow & \downarrow \\ 0010 & 1010 & 1111 \end{array}$$

$$(2AF)_{16} = 0010\ 1010\ 1111_2$$

**Example:**  $(7C3)_{16}$  to binary

$$\begin{array}{ccc} 7 & C & 3 \\ \downarrow & \downarrow & \downarrow \\ 0111 & 1100 & 0011 \end{array}$$

$$(7C3)_{16} = 0111\ 1100\ 0011_2$$

### 6. Binary to Hexadecimal Conversion

**Steps:**

- Group binary digits into sets of 4 bits from right side (add leading zeros if needed).
- Convert each 4-bit group to its hex equivalent.

**Example 1:**  $10101110001_2$  to hexadecimal

$$\begin{array}{ccc} 1010 & 1111 & 0001 \\ \downarrow & \downarrow & \downarrow \\ A & F & 1 \end{array}$$

$$10101110001_2 = AF1_{16}$$

**Example 2:**  $1110\ 0101\ 0011_2$  to hexadecimal

$$\begin{array}{ccc} 1110 & 0101 & 0011 \\ \downarrow & \downarrow & \downarrow \\ E & 5 & 3 \end{array}$$

$$111001010011_2 = E53_{16}$$

### 7. Hexadecimal to Octal Conversion

**Method:** Hex  $\rightarrow$  Binary  $\rightarrow$  Octal

**Example:**  $(3F)_{16}$  to octal

$$(3F)_{16} = 37_8$$

**Step 1:** Hex to binary

$$3 \rightarrow 0011 \quad F \rightarrow 1111$$

$$\text{Binary} = 0011\ 1111$$

**Step 2:** Group into 3 bits (from right)

$$\begin{array}{ccc} 00 & 011 & 111 \\ \downarrow & \downarrow & \downarrow \\ 0 & 3 & 7 \end{array}$$

### 8. Octal to Hexadecimal Conversion

**Method:** Octal  $\rightarrow$  Binary  $\rightarrow$  Hex

**Example:**  $(725)_8$  to hexadecimal

**Step 1:** Octal to binary

$$7 \rightarrow 111 \quad 2 \rightarrow 010 \quad 5 \rightarrow 101$$

$$\text{Binary} = 11\ 010\ 101$$

**Step 2:** Group into 4 bits (from right)

$$\begin{array}{ccc} 0001 & 1101 & 0101 \\ \downarrow & \downarrow & \downarrow \\ 1 & D & 5 \end{array}$$

$$(725)_8 = 1D5_{16}$$

### 9. Quick Reference Table

| Hex | Dec | Binary | Hex | Dec | Binary |
|-----|-----|--------|-----|-----|--------|
| 0   | 0   | 0000   | 8   | 8   | 1000   |
| 1   | 1   | 0001   | 9   | 9   | 1001   |
| 2   | 2   | 0010   | A   | 10  | 1010   |
| 3   | 3   | 0011   | B   | 11  | 1011   |
| 4   | 4   | 0100   | C   | 12  | 1100   |
| 5   | 5   | 0101   | D   | 13  | 1101   |
| 6   | 6   | 0110   | E   | 14  | 1110   |
| 7   | 7   | 0111   | F   | 15  | 1111   |

"Different bases, same number, different view!"



Same Rules...  
Different Base !



# BINARY ARITHMETIC

## Addition and Subtraction using 1's & 2's Complements

"Complement makes  
Subtraction as easy as  
Addition !" 😊

### 1. Compliments - Basics

- 1's Complement:** Replace each 0 by 1 and each 1 by 0.
- 2's Complement:** Add 1 to the 1's complement.
- Used to represent negative numbers in binary.
- For an n-bit number, range using 2's complement:  $-2^{n-1}$  to  $+2^{n-1} - 1$

### 2. Finding 1's and 2's Complement

#### Steps:

- For 1's complement → invert all bits.
- For 2's complement → add 1 to the 1's complement.

**Example:** (8-bit)

Find 1's and 2's complement of 01011001

#### Solution:

1's complement: 10100110  
2's complement: 10100111

### 3. Properties

- Double complement gives the original number.
- A number + its 1's complement = all 1's (end-around carry).
- A number + its 2's complement = 0 (ignoring carry).
- 2's complement is widely used for subtraction and in computers.

### 4. Addition using 1's Complement

#### Steps:

- Add the two numbers.
- If there is a carry out, add the carry to the least significant bit (end-around carry).

**Example 1:** Add 10110101 and 01101111 using 1's complement

#### Solution:

```

10110101
+ 01101111
-----

```

100100100  
(carry out = 1)

Add carry to LSB:  
00100100 + 1  
00100101

**Answer = 00100101**

In 1's complement addition, if there is a carry, add it to the result (end-around carry).

### 7. Subtraction using 2's Complement

#### Steps:

- Subtract A - B as A + (2's complement of B).
- Ignore the carry out.
- If the result is negative (no carry out), take 2's complement and put a minus sign.

**Example 4:** Subtract 01001000 - 00101101 using 2's complement

#### Solution:

2's complement of 00101101:  
1's complement = 11010010  
Add 1 = 11010011

```

01001000
+ 11010011
-----

```

1 00111011 (carry out = 1, ignore)

**Answer = 00111011**

### 5. Addition using 2's Complement

#### Steps:

- Add the two numbers.
- Ignore the carry out (if any).

**Example 2:** Add 11010110 and 01101101 using 2's complement

#### Solution:

```

11010110
+ 01101101
-----

```

1 01000111 (carry out = 1, ignore in 2's complement)

**Answer = 01000111**

#### Note:

In 2's complement addition, carry out is ignored.

### 8. Subtraction with Negative Result (2's Complement)

**Example 5:** Subtract 01010100 - 11001000

#### Solution:

2's complement of 11001000:  
1's complement = 00110111  
Add 1 = 00111000

```

01010100
+ 00111000
-----

```

100011100 (no carry out)

Result is negative.

Take 2's complement of 00011100:

1's complement = 11100011  
Add 1 = 11100100  
So, answer = -11100100

If no carry is generated, the result is negative.  
Take 2's complement and add (-) sign.

### 6. Subtraction using 1's Complement

#### Steps:

- Subtract A - B as A + (1's complement of B).
- If there is a carry out, add it to the result (end-around carry).

**Example 3:** Subtract 11001010 - 01100111 using 1's complement

#### Solution:

1's complement of 01100111 = 10011000

```

11001010
+ 10011000
-----

```

1 01100010 (carry out = 1)

Add carry to LSB:

```

01100010 + 1
01100011

```

If carry is generated, add it to the result.

**Answer = 01100011**

### 9. Quick Examples

- Add using 1's complement: 11101010 + 00011101
- Add using 2's complement: 10100101 + 01010111
- Subtract using 1's complement: 10011000 - 01100101
- Subtract using 2's complement: 01100011 - 11101010

### 10. Key Points to Remember

- ✓ 1's complement → invert all bits.
- ✓ 2's complement → add 1 to 1's complement.
- ✓ In 1's complement addition, add end-around carry.
- ✓ In 2's complement addition, ignore carry out.
- ✓ If no carry in subtraction (2's complement), result is negative.
- ✓ 2's complement is the most commonly used method in computers.

"Bits today,  
Bright tomorrow !" 😊



Same Rules,  
Different Base  
(Just 0 and 1)!

# BINARY ARITHMETIC

## Multiplication & Division

"Practice Today"  
for a Brighter  
Tomorrow!

### BINARY MULTIPLICATION

#### 1. Basic Rules

- Multiplication is similar to decimal multiplication.
- Only 0 and 1 are used.
- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

Multiply and add  
partial products.

#### 3. Example 1: Multiply $1011_2 \times 110_2$

Multiplicand =  $1011$  ( $11_{10}$ )  
Multiplier =  $110$  ( $6_{10}$ )  
Product = ?

Solution:

$$\begin{array}{r} 1011 \\ \times 110 \\ \hline 0000 \quad (1011 \times 0) \\ 1011 \quad (1011 \times 1) \\ 11110 \quad (1011 \times 1, \text{shifted left}) \\ \hline 1000000 \end{array}$$

$$1011_2 \times 110_2 = 1000010_2$$

$$(11_{10} \times 6_{10} = 66_{10})$$

#### 2. Steps

- Write the numbers (multiplicand and multiplier) in binary.
- Multiply the multiplicand with each bit of the multiplier (from right to left).
- Write partial products.
- Shift each partial product to the left according to the bit position.
- Add all partial products.

#### 4. Example 2: Multiply $1110_2 \times 101_2$

Multiplicand =  $1110$  ( $14_{10}$ )  
Multiplier =  $101$  ( $5_{10}$ )  
Product = ?

Solution:

$$\begin{array}{r} 1110 \\ \times 101 \\ \hline 1110 \quad (1110 \times 1) \\ 0000 \quad (1110 \times 0, \text{shifted left}) \\ 11100 \quad (1110 \times 1, \text{shifted left}) \\ \hline 100010 \end{array}$$

$$1110_2 \times 101_2 = 100010_2$$

$$(14_{10} \times 5_{10} = 70_{10})$$

#### 5. Key Points (Multiplication)

- ✓ Procedure is same as decimal multiplication.
- ✓ Each partial product is either the multiplicand (for 1) or 0 (for 0).
- ✓ Shift left for each next bit.
- ✓ Add all partial products to get the final result.

### BINARY DIVISION

#### 1. Basic Rules

- Division is similar to decimal long division.
- Subtract (using 2's complement or direct subtraction) and bring down the next bit.
- Only 0 and 1 are used.
- At each step, check if divisor can be subtracted from the current left bits.

#### 3. Example 1: Divide $110110_2$ by $101_2$

Dividend =  $110110$  ( $54_{10}$ )  
Divisor =  $101$  ( $5_{10}$ )  
Quotient = ?  
Remainder = ?

Solution:

$$\begin{array}{r} 510 \rightarrow \text{Quotient } (110_2) \\ 101 \overline{) 110110} \\ \underline{101} \phantom{00} \\ 0111 \phantom{0} \\ \underline{000} \phantom{0} \\ 1110 \phantom{0} \\ \underline{1010} \phantom{0} \\ 100 \rightarrow \text{Remainder } (100_2) \end{array}$$

$$110110_2 \div 101_2 = 110_2 \quad R = 100_2$$

$$(54_{10} \div 5_{10} = 11_{10} \quad R = 4_{10})$$

#### 2. Steps (Long Division Method)

- Write the dividend and divisor.
- Compare the leftmost bits with the divisor.
- If divisor  $\leq$  current bits  $\rightarrow$  write 1 in quotient and subtract.
- If divisor  $>$  current bits  $\rightarrow$  write 0 in quotient and bring down next bit.
- Repeat until all bits are used.

#### 4. Example 2: Divide $10001_2$ by $11_2$

Dividend =  $10001$  ( $17_{10}$ )  
Divisor =  $11$  ( $3_{10}$ )  
Quotient = ?  
Remainder = ?

Solution:

$$\begin{array}{r} 710 \rightarrow \text{Quotient } (710_2) \\ 11 \overline{) 10001} \\ \underline{11} \phantom{000} \\ 010 \phantom{0} \\ \underline{00} \phantom{0} \\ 100 \phantom{0} \\ \underline{11} \phantom{0} \\ 01 \rightarrow \text{Remainder } (01_2) \end{array}$$

$$10001_2 \div 11_2 = 710_2 \quad R = 01_2$$

$$(17_{10} \div 3_{10} = 5_{10} \quad R = 2_{10})$$

#### 5. Key Points (Division)

- ✓ Use long division method.
- ✓ At each step, compare and subtract.
- ✓ If subtraction is not possible, write 0 in the quotient and bring down the next bit.
- ✓ The final result gives quotient and remainder.
- ✓ Remainder is always less than the divisor.

Keep Practicing  
Keep Growing!

Each decimal digit is stored separately in 4 bits (8421)!



# 8421 BCD (Binary Coded Decimal)

## Represent Decimal Digits in Binary Form

Same number...  
Different codes...  
Same value! 😊

### 1. What is 8421 BCD?

- BCD (Binary Coded Decimal) is a code used to represent decimal digits (0-9) in binary form.
- In 8421 BCD, each decimal digit is represented by a 4-bit binary number with weights 8, 4, 2, 1.
- It is called 8421 because the bit positions have weights 8, 4, 2 and 1 (from left to right).
- Each decimal digit is coded separately (so 2-digit number uses 8 bits).
- Widely used in digital electronics, calculators, digital displays, and computer systems.

### 2. Bit Weights and Code Table

| Bit Position  | MSB             |   |               |                 | LSB |
|---------------|-----------------|---|---------------|-----------------|-----|
|               | 8               | 4 | 2             | 1               |     |
| Decimal Digit | BCD Code (8421) |   | Decimal Digit | BCD Code (8421) |     |
| 0             | 0000            |   | 5             | 0101            |     |
| 1             | 0001            |   | 6             | 0110            |     |
| 2             | 0010            |   | 7             | 0111            |     |
| 3             | 0011            |   | 8             | 1000            |     |
| 4             | 0100            |   | 9             | 1001            |     |

**Note:** 1010 (10) to 1111 (15) are invalid in BCD (since decimal digits are only 0-9).

### 3. Converting Decimal to 8421 BCD

#### Steps:

- Write each decimal digit.
- Replace each digit with its 4-bit BCD code (from the table).
- Combine all the groups.

**Example 1:** Convert (59)<sub>10</sub> to BCD

5 → 0101  
9 → 1001

(59)<sub>10</sub> = 0101 1001 (BCD)

**Example 2:** Convert (275)<sub>10</sub> to BCD

2 → 0010  
7 → 0111  
5 → 0101

(275)<sub>10</sub> = 0010 0111 0101 (BCD)

### 4. Converting 8421 BCD to Decimal

#### Steps:

- Split the code into 4-bit groups (from left to right).
- Convert each group to its decimal digit using the table.
- Combine the digits to get the decimal number.

**Example 3:** Convert 0101 1001 (BCD) to decimal

0101 → 5  
1001 → 9

Decimal number = (59)<sub>10</sub>

**Example 4:** Convert 0010 0111 0101 (BCD) to decimal

0010 → 2  
0111 → 7  
0101 → 5

Decimal number = (275)<sub>10</sub>

### 5. BCD Addition (Using Decimal Correction)

- Add the BCD numbers like normal binary.
- If the 4-bit sum is > 9 (i.e., 1010 to 1111) or there is a carry, add 0110 (decimal 6) to correct it.

**Example 5:** Add (37)<sub>10</sub> and (45)<sub>10</sub> using BCD

|                           |      |                                                  |
|---------------------------|------|--------------------------------------------------|
| 3                         | 7    |                                                  |
| ↓                         | ↓    |                                                  |
| 0011                      | 0111 | (value)                                          |
| (37)                      | (45) |                                                  |
| +                         |      |                                                  |
| 0111                      | 1100 | (Initial Sum)                                    |
| +                         |      |                                                  |
| 0111                      | 1100 | (Correction: Lower nibble = 1100 (> 9) Add 0110) |
| 1000 0010 (Corrected Sum) |      |                                                  |

**Result = (82)<sub>10</sub>**

### 6. BCD Subtraction

- Subtract the numbers like normal binary (using 10's complement or borrow method).
- If the result in any nibble is invalid (borrow occurs), add 0110 to correct it.

**Example 6:** Subtract (58)<sub>10</sub> - (23)<sub>10</sub> using BCD

|      |      |      |      |
|------|------|------|------|
| 5    | 8    | 2    | 3    |
| ↓    | ↓    | ↓    | ↓    |
| 0101 | 1000 | 0010 | 0011 |
| (58) |      | (02) | (21) |
|      |      | -    |      |
|      |      | 0011 | 0101 |

**Result = (35)<sub>10</sub>**

If borrow occurs, add 0110 to the corresponding nibble.

### 7. Important Points

- ✓ Each decimal digit is represented by 4 bits.
- ✓ Only digits 0-9 are valid.
- ✓ 1010 to 1111 are invalid in 8421 BCD.
- ✓ Used in calculators, digital clocks, computers, etc.
- ✓ Addition and subtraction in BCD require correction (add 0110).
- ✓ BCD is not a weight-based code for the entire number, only for individual digits.
- ✓ Easy to convert since each digit is coded separately.
- ✓ Useful for display devices (e.g., 7-segment display).

### 8. Applications

- Digital calculators
- Digital clocks and watches
- 7-segment displays
- Data input/output systems
- Computer internal processing
- Financial and commercial systems

8421 BCD  
Simple Code...  
Big Use! 😊



Codes connect numbers  
to a digital world!



Same Number...  
Different Codes...  
Different Uses! 😊

# BINARY CODES

## Excess-3 Code & Gray Code

Codes make  
digital systems  
smarter! 💡

### EXCESS-3 CODE (XS-3)

#### 1. What is Excess-3 Code?

- Excess-3 (XS-3) code is a non-weighted code.
- It is obtained by adding 3 (0011) to each decimal digit and then writing the result in 4-bit binary form.
- It is also called "XS-3 Code".
- Used for error detection and in some digital systems (e.g., calculators).

#### 3. Example: Decimal to Excess-3

Convert  $(59)_{10}$  to Excess-3 code.

Step 1: Write each digit and add 0011.

Step 2: Combine the codes.

5 → 1000  
9 → 1100

$(59)_{10} = 1000\ 1100$  (Excess-3)

#### 4. Decoding Table (Excess-3 to Decimal)

| Excess-3 Code | Decimal Digit | Excess-3 Code | Decimal Digit |
|---------------|---------------|---------------|---------------|
| 0011          | 0             | 1000          | 5             |
| 0100          | 1             | 1001          | 6             |
| 0101          | 2             | 1010          | 7             |
| 0110          | 3             | 1011          | 8             |
| 0111          | 4             | 1100          | 9             |

#### 6. Key Points

- ✓ Excess-3 code is obtained by adding 3 to each decimal digit.
- ✓ It is a non-weighted code.
- ✓ Each decimal digit is represented by a 4-bit code.
- ✓ Codes range from 0011 (0) to 1100 (9).
- ✓ Used in digital calculators, display systems, and for error detection.

#### 2. Encoding Table (Decimal to Excess-3)

| Decimal Digit | Binary | Add 0011 | Excess-3 Code |
|---------------|--------|----------|---------------|
| 0             | 0000   | 0011     | 0011          |
| 1             | 0001   | 0011     | 0100          |
| 2             | 0010   | 0011     | 0101          |
| 3             | 0011   | 0011     | 0110          |
| 4             | 0100   | 0011     | 0111          |
| 5             | 0101   | 0011     | 1000          |
| 6             | 0110   | 0011     | 1001          |
| 7             | 0111   | 0011     | 1010          |
| 8             | 1000   | 0011     | 1011          |
| 9             | 1001   | 0011     | 1100          |

#### 5. Example: Excess-3 to Decimal

Convert 0110 1001 to decimal.

0110 → 3  
1001 → 6

$(36)_{10}$

Excess-3  
= Decimal + 3

### GRAY CODE (REFLECTED BINARY CODE)

#### 1. What is Gray Code?

- Gray code is a non-weighted binary code.
- Also called "Reflected Binary Code".
- Successive codes differ by only one bit (Hamming distance = 1).
- Used to minimize errors in digital systems (e.g., encoders, position sensors, A/D converters).

#### 3. How to Convert Binary to Gray Code?

Steps:

- Keep the MSB (leftmost bit) same.
- For each next bit, perform XOR with the previous binary bit.

$$G_n = B_n \quad (\text{MSB same})$$

$$G_i = B_i \oplus B_{i-1} \quad (\text{for } i = n-1, n-2, \dots, 0)$$

Example 1: Convert binary 1011 to Gray code.

Binary : 1 0 1 1

Gray : 1 1 1 0

Gray code = 1110

$$\begin{cases} G_3 = 1 \\ G_2 = 1 \oplus 0 = 1 \\ G_1 = 0 \oplus 1 = 1 \\ G_0 = 1 \oplus 1 = 0 \end{cases}$$

#### 4. How to Convert Gray Code to Binary?

Steps:

- Keep the MSB same.
- For each next bit, perform XOR with the previous binary bit (not previous Gray bit).

Example 2: Convert Gray code 1101 to binary.

Gray : 1 1 0 1

Binary : 1 0 1 0

Binary code = 1010

$$\begin{cases} B_3 = 1 \\ B_2 = 1 \oplus 1 = 0 \\ B_1 = 0 \oplus 0 = 0 \\ B_0 = 1 \oplus 0 = 1 \end{cases}$$

#### 2. 4-Bit Gray Code Table

| Decimal Digit | Binary Code | Gray Code |
|---------------|-------------|-----------|
| 0             | 0000        | 0000      |
| 1             | 0001        | 0001      |
| 2             | 0010        | 0011      |
| 3             | 0011        | 0010      |
| 4             | 0100        | 0110      |
| 5             | 0101        | 0111      |
| 6             | 0110        | 0101      |
| 7             | 0111        | 0100      |
| 8             | 1000        | 1100      |
| 9             | 1001        | 1101      |
| 10            | 1010        | 1111      |
| 11            | 1011        | 1110      |
| 12            | 1100        | 1010      |
| 13            | 1101        | 1011      |
| 14            | 1110        | 1001      |
| 15            | 1111        | 1000      |

#### 5. Key Points

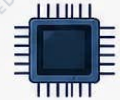
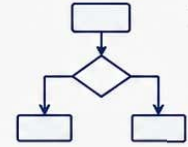
- ✓ Successive Gray codes differ by only one bit.
- ✓ Minimizes errors in digital systems.
- ✓ Used in position encoders, A/D converters, and communication systems.
- ✓ It is a non-weighted code.
- ✓ It is a cyclic code (last and first codes differ by one bit).

#### 6. Applications

- Rotary and linear encoders
- Analog to Digital Converters (ADC)
- Error minimization in digital systems
- Flash counters
- Robotics and industrial automation

Learn  
Practice  
Apply  
Grow! 😊

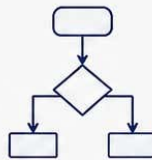
1010101010  
0101010101  
1010101010  
0101010101



# Unit - II



10110  
01001  
10101



Input + Logic  
= Output  
Simple but Powerful! 😊

# LOGIC GATES

Building Blocks of Digital Electronics

"Small Gates  
Create Big Possibilities!" 😊

Digital  
Electronics

## 1. What are Logic Gates?

- A logic gate is a basic electronic circuit that performs a logical operation on one or more binary inputs to produce a single binary output.
- Inputs and outputs are in binary form (0 or 1).
- Logic gates are the fundamental building blocks of digital systems such as computers, calculators, and communication devices.
- They are implemented using diodes, transistors, or ICs.

Binary Input (0 or 1)

↓  
Logic Gate

↓  
Binary Output (0 or 1)

## 2. Basic Logic Gates: Definition, Symbol and Truth Table

### AND Gate (•)

Output is 1 only when all inputs are 1.



Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

$$Y = A \cdot B$$

### OR Gate (+)

Output is 1 when any one (or both) inputs are 1.



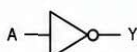
Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

$$Y = A + B$$

### NOT Gate (Inverter)

Output is the complement of input.



Truth Table

| A | Y |
|---|---|
| 0 | 1 |
| 1 | 0 |

$$Y = \bar{A}$$

### NAND Gate

Output is 0 only when all inputs are 1.  
(Complement of AND)



Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

$$Y = \overline{A \cdot B}$$

### NOR Gate

Output is 1 only when all inputs are 0.  
(Complement of OR)



Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

$$Y = \overline{A + B}$$

### EX-OR Gate (XOR)

Output is 1 when inputs are different.



Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

$$Y = A \oplus B$$

### EX-NOR Gate (XNOR)

Output is 1 when inputs are same.



Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

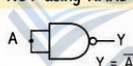
$$Y = \overline{A \oplus B}$$

## 3. Universal Gates

- NAND and NOR gates are called Universal Gates because any logic gate (AND, OR, NOT, etc.) can be implemented using only NAND or only NOR gates.
- They are widely used in digital IC design.

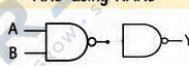
## 4. Realization of Basic Gates using NAND

### NOT using NAND



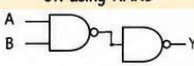
(Both inputs same)  
 $Y = \bar{A}$

### AND using NAND



$$Y = A \cdot B$$

### OR using NAND



$$Y = A + B$$

## 5. Realization of Basic Gates using NOR

### NOT using NOR



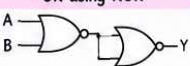
(Both inputs same)  
 $Y = \bar{A}$

### AND using NOR



$$Y = A \cdot B$$

### OR using NOR



$$Y = A + B$$

## 6. Comparison Table

| Gate | Symbol | Output Condition        | Expression                 |
|------|--------|-------------------------|----------------------------|
| AND  |        | 1 when all inputs 1     | $Y = A \cdot B$            |
| OR   |        | 1 when any input 1      | $Y = A + B$                |
| NOT  |        | Complement of input     | $Y = \bar{A}$              |
| NAND |        | 0 when all inputs 1     | $Y = \overline{A \cdot B}$ |
| NOR  |        | 1 when all inputs 0     | $Y = \overline{A + B}$     |
| XOR  |        | 1 when inputs different | $Y = A \oplus B$           |
| XNOR |        | 1 when inputs same      | $Y = A \ominus B$          |

## 7. Important Points

- ✓ Inputs and outputs are binary (0 or 1).
- ✓ NAND and NOR are universal gates.
- ✓ NOT gate is also called an inverter.
- ✓ AND performs logical multiplication.
- ✓ OR performs logical addition.
- ✓ XOR is used for comparison.
- ✓ XNOR gives 1 when inputs are same.
- ✓ Logic gates are implemented using transistors/ICs.
- ✓ Used in all digital systems.

## 8. Applications

- ✓ Digital computers
- ✓ Calculators
- ✓ Data processing
- ✓ Communication systems
- ✓ Control systems
- ✓ Memory circuits
- ✓ Arithmetic and logic unit (ALU)
- ✓ Design of combinational and sequential circuits

"Logic Gates  
are the language of  
Digital Computers!" 😊

## 9. Example: Evaluate the Output

Find the output Y for the given inputs (A = 0, B = 1)

| Gate | Expression                 | Output |
|------|----------------------------|--------|
| AND  | $Y = A \cdot B$            | 0      |
| OR   | $Y = A + B$                | 1      |
| NAND | $Y = \overline{A \cdot B}$ | 1      |
| NOR  | $Y = \overline{A + B}$     | 0      |
| XOR  | $Y = A \oplus B$           | 1      |
| XNOR | $Y = A \ominus B$          | 0      |

Learn  
Practice  
Build  
Succeed! 😊

Digital Electronics

"Logic Gates Turn Ideas into Intelligent Machines!"

Single Input ...  
Opposite Output ! 😊

# NOT GATE (INVERTER)

Definition, Symbol, Truth Table and Working

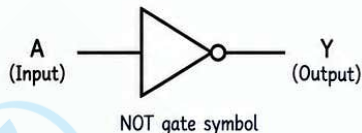
"Change 0 to 1  
and 1 to 0 !"  
That's a NOT Gate 😊

## 1. Definition

- A NOT gate is a basic logic gate that has only **one input** and **one output**.
- It produces the **complement** (inverse) of the input.
- If the input is 0, the output is 1, and if the input is 1, the output is 0.
- It is also called an **Inverter** because it inverts the input signal.

## 2. Symbol

Logic Symbol:



The small circle (bubble) at the output represents inversion (complement).

## 3. Boolean Expression

$$Y = \bar{A}$$

Where,

- A = Input
- Y = Output (Complement of A)

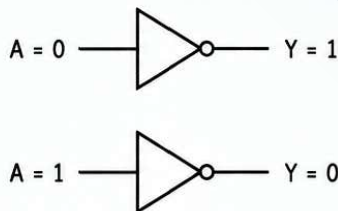
Read as:  
Y is NOT A  
or  
Y is the complement of A.

## 4. Truth Table

| Input<br>A | Output<br>Y = $\bar{A}$ |
|------------|-------------------------|
| 0          | 1                       |
| 1          | 0                       |

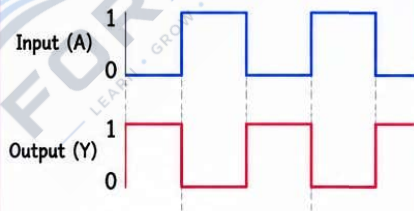
When input is 0 → output is 1  
When input is 1 → output is 0

## 5. Circuit Example



The output is always the opposite of the input.

## 6. Waveform Representation



Output waveform is the inverse of the input waveform.

## 7. Key Points

- ✓ NOT gate has only one input and one output.
- ✓ It gives the complement of the input.
- ✓ Also called an **Inverter**.
- ✓ Symbol is a triangle with a small circle at the output.
- ✓ Boolean expression:  $Y = \bar{A}$
- ✓ Used in many digital circuits, computers and control systems.
- ✓ It is a basic building block for more complex logic circuits.

## 8. Applications

- Signal inversion
- Used in data processing
- In memory circuits
- In digital computers
- In control systems
- In combination with other gates to design complex circuits (e.g., NAND, NOR, etc.)

Learn  
Practice  
Build  
Succeed



Digital Electronics

"Sometimes, the opposite is exactly what you need !" 😊

If any input is 1  
→ Output is 1 ! 😊

# OR GATE (INCLUSIVE OR)

Definition, Symbol, Truth Table and Applications

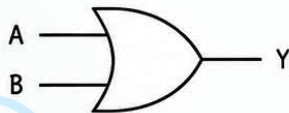
Simple Gate ...  
Big Possibilities ! 😊

## 1. Definition

- An OR gate is a basic logic gate that performs the logical OR operation.
- It has two or more inputs and **one output**.
- The output is 1 (HIGH) if **any one or more of the inputs are 1**.
- The output is 0 (LOW) only when all inputs are 0.
- It is called an **Inclusive OR gate** because the output is 1 when **at least one input is 1** (including the case when both inputs are 1).

## 2. Symbol

OR Gate Symbol (2 Inputs)



The OR gate symbol has a curved shape with two (or more) inputs and one output.

## 3. Boolean Expression

$$Y = A + B$$

Where,

- A = Input 1
- B = Input 2
- Y = Output
- "+" denotes the OR operation (logical addition).

Read as:  
Y is A **OR** B

## 4. Truth Table

| A<br>(Input 1) | B<br>(Input 2) | Y = A + B<br>(Output) |
|----------------|----------------|-----------------------|
| 0              | 0              | 0                     |
| 0              | 1              | 1                     |
| 1              | 0              | 1                     |
| 1              | 1              | 1                     |

The output is 1 if any input is 1.  
The output is 0 only when all inputs are 0.

## 5. Example

Find the output Y for the following inputs using OR gate.

| A | B | Y = A + B |
|---|---|-----------|
| 0 | 0 | 0         |
| 0 | 1 | 1         |
| 1 | 0 | 1         |
| 1 | 1 | 1         |

## 6. Real-Life Analogy



A light turns ON (1) if any one of the switches is ON (1).  
It turns OFF (0) only when both switches are OFF (0).

## 7. Key Points

- ✓ OR gate performs logical addition (OR operation).
- ✓ Output is 1 if any input is 1.
- ✓ Output is 0 only when all inputs are 0.
- ✓ It is an Inclusive OR gate.
- ✓ Used to combine multiple signals.
- ✓ The symbol has a curved shape.
- ✓ Can have two or more inputs.
- ✓ Widely used in digital circuits, control systems and decision making logic.

## 8. Applications

- Used in digital computers
- In alarm systems (any sensor active gives an alarm)
- In control circuits
- In communication systems
- In decision making logic
- In arithmetic circuits
- In combination with other gates to design complex digital systems

Learn

Practice

Apply

Succeed

"OR Gate - Because one good reason is enough !" 😊

Both inputs must be 1 to get 1 at output ! 😊

# AND GATE (LOGICAL MULTIPLICATION)

Definition, Symbol, Truth Table, Working and Applications

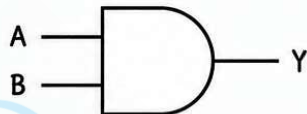
Simple Gate...  
Powerful Use ! 😊

## 1. Definition

- An AND gate is a basic logic gate that performs the **logical AND** operation.
- It has two or more inputs and one output.
- The output is 1 (HIGH) only when **all the inputs are 1**.
- If any input is 0, the output is 0 (LOW).
- The AND gate is also called a **logical multiplier** because it gives 1 only when all conditions are satisfied.

## 2. Symbol

AND Gate Symbol (2 Inputs)



The AND gate symbol has a flat left side and a curved right side with two (or more) inputs and one output.

## 3. Boolean Expression

$$Y = A \cdot B$$

Where,

- A = Input 1
- B = Input 2
- Y = Output
- " $\cdot$ " (dot) denotes the AND operation (logical multiplication).

Read as:

Y is A AND B

(Y is 1 only if both A and B are 1)

## 4. Truth Table

| A<br>(Input 1) | B<br>(Input 2) | Y = A · B<br>(Output) |
|----------------|----------------|-----------------------|
| 0              | 0              | 0                     |
| 0              | 1              | 0                     |
| 1              | 0              | 0                     |
| 1              | 1              | 1                     |

The output is 1 only when both inputs are 1.

## 5. Logic Example

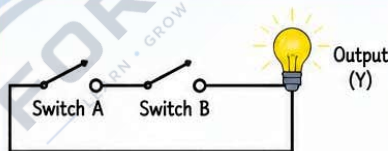
Find the output Y for the following inputs using AND gate.

| A | B | Y = A · B |
|---|---|-----------|
| 0 | 0 | 0         |
| 0 | 1 | 0         |
| 1 | 0 | 0         |
| 1 | 1 | 1         |

Example:

If A = 1 and B = 1, then Y = 1.

## 6. Real-Life Analogy



A bulb will glow (Y = 1) only when both switches A and B are ON (1). If any one switch is OFF (0), the bulb will not glow (Y = 0).

## 7. Key Points

- ✓ Performs logical AND operation.
- ✓ Output is 1 only when all inputs are 1.
- ✓ Output is 0 if any input is 0.
- ✓ Also called a logical multiplier.
- ✓ Symbol has a flat left side and a curved right side.
- ✓ Can have two or more inputs.
- ✓ Used in digital circuits, control systems and decision making.
- ✓ Widely used in combination with other logic gates.

## 8. Applications

- Used in digital computers (arithmetic and control units).
- In security systems (e.g., multiple conditions must be satisfied).
- In control circuits.
- In alarm systems.
- In data processing.
- In communication systems.
- In combination with other gates to design complex logic circuits.

Think Logic  
Build Solutions ! 😊

"AND Gate – All Conditions Must Be True !"



Combination of  
AND + NOT ! 😊

# NAND GATE (NOT-AND)

Definition, Symbol, Truth Table, Working and Applications

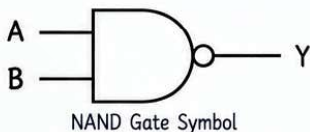
Universal Gate  
(We can build any  
logic circuit using NAND) 😊

## 1. Definition

- A NAND gate is a basic logic gate that performs the **complement** of the **AND operation**.
- It has two or more inputs and one output.
- The output is 0 (LOW) only when all the inputs are 1 (HIGH).
- For all other input combinations, the output is 1 (HIGH).
- It is called a NAND gate because it is an AND gate followed by a NOT gate (inverter).
- NAND gate is also known as a **universal gate** since all other logic gates can be implemented using only NAND gates.

## 2. Symbol

NAND Gate Symbol (2 Inputs)



The NAND gate symbol is the same as the AND gate symbol with a small circle (inversion bubble) at the output.

General Symbol (n inputs)



## 3. Boolean Expression

$$Y = (A \cdot \bar{B})$$

Where,

- A = Input 1
- B = Input 2
- Y = Output
- (dot) represents AND operation
- ( $\bar{\phantom{x}}$ ) represents complementation (NOT)

Alternative form:

$$Y = \bar{A} + \bar{B}$$

(By De Morgan's Theorem)

## 4. Truth Table

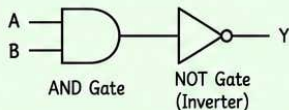
| A<br>(Input 1) | B<br>(Input 2) | Y = (A · $\bar{B}$ )<br>(Output) |
|----------------|----------------|----------------------------------|
| 0              | 0              | 1                                |
| 0              | 1              | 1                                |
| 1              | 0              | 1                                |
| 1              | 1              | 0                                |

Output is 0 only when all inputs are 1.  
For all other combinations, output is 1.

## 5. Working

- First, the inputs are applied to an AND gate.
- The output of the AND gate is then inverted using a NOT gate.
- So, the NAND gate gives the opposite (complement) of the AND operation.

Equivalent Circuit



## 6. Example

Find the output Y for the following inputs:

| A | B | Y = (A · $\bar{B}$ ) |
|---|---|----------------------|
| 0 | 0 | 1                    |
| 0 | 1 | 1                    |
| 1 | 0 | 1                    |
| 1 | 1 | 0                    |

**Example:**

If A = 1 and B = 0, then Y = 1  
(because  $1 \cdot 0 = 0$  and its complement is 1).

## 7. Key Points

- ✓ NAND gate performs the NOT-AND operation.
- ✓ Output is 0 only when all inputs are 1.
- ✓ Output is 1 for all other input combinations.
- ✓ Boolean expression:  $Y = (A \cdot \bar{B})$
- ✓ Equivalent expression:  $Y = \bar{A} + \bar{B}$
- ✓ Symbol is an AND gate with a small circle at the output.
- ✓ It is a universal gate (all other gates can be built using NAND).
- ✓ Widely used in digital ICs and logic design.

## 8. Applications

- Used in digital computers.
- Used in control systems.
- Used in memory circuits (e.g., SRAM, flip-flops).
- Used in logic design (to implement other gates like AND, OR, NOT, etc.).
- Used in arithmetic circuits.
- Used in communication systems.
- Used in embedded systems and VLSI design.



Digital Electronics

"NAND Gate – A Small Gate with Big Possibilities !"



Build Logic  
with NAND ! 😊

Output is 1 only when all inputs are 0! 😊

# NOR GATE (NOT-OR)

Definition, Symbol, Truth Table, Working and Applications

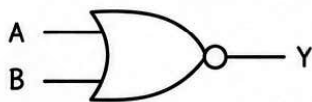
NOR Gate = OR + NOT  
A Universal Gate! 😊

## 1. Definition

- A NOR gate is a basic logic gate that performs the **complement of the OR operation**.
- It has two or more inputs and one output.
- The output is 1 (HIGH) only when all the inputs are 0 (LOW).
- If any input is 1, the output is 0.
- It is called a NOR gate because it is an OR gate followed by a NOT gate (inverter).
- NOR gate is also a **universal gate**, meaning any other logic gate can be implemented using only NOR gates.

## 2. Symbol

NOR Gate Symbol (2 Inputs)



NOR Gate Symbol

It is an OR gate symbol with a small circle (inversion bubble) at the output to indicate complementation.

General Symbol (n inputs)



## 3. Boolean Expression

$$Y = \overline{A + B}$$

Where,

- A = Input 1
- B = Input 2
- Y = Output
- “+” denotes the OR operation (logical addition).
- The overbar indicates complementation (NOT).

Read as:

Y is NOT (A OR B)  
or

Y is the complement of A OR B

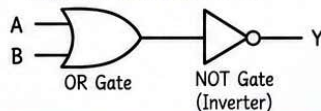
## 4. Truth Table

| A<br>(Input 1) | B<br>(Input 2) | Y = $\overline{A + B}$<br>(Output) |
|----------------|----------------|------------------------------------|
| 0              | 0              | 1                                  |
| 0              | 1              | 0                                  |
| 1              | 0              | 0                                  |
| 1              | 1              | 0                                  |

The output is 1 only when all inputs are 0.  
For all other combinations, the output is 0.

## 5. Working

- The inputs are first applied to an OR gate.
- The OR gate gives an output of 1 if any input is 1.
- This output is then inverted using a NOT gate.
- Therefore, the final output is 1 only when all inputs are 0.



## 6. Example

Find the output Y for the following inputs using NOR gate.

| A | B | Y = $\overline{A + B}$ |
|---|---|------------------------|
| 0 | 0 | 1                      |
| 0 | 1 | 0                      |
| 1 | 0 | 0                      |
| 1 | 1 | 0                      |

Example:

If A = 0 and B = 0, then Y = 1.

If A = 1 and B = 0, then Y = 0  
(because  $1 + 0 = 1$ , and its complement is 0).

## 7. Key Points

- ✓ NOR gate performs the NOT-OR operation.
- ✓ Output is 1 only when all inputs are 0.
- ✓ Output is 0 if any input is 1.
- ✓ Boolean expression:  $Y = \overline{A + B}$
- ✓ Symbol is an OR gate with a small circle at the output.
- ✓ It is a universal gate (all other gates can be built using NOR).
- ✓ Widely used in digital circuits and logic design.
- ✓ Also called a complemented OR gate.

## 8. Applications

- Used in digital computers.
- Used in control systems.
- Used in memory circuits (e.g., NOR-based SRAM).
- Used in logic design (to implement other gates like AND, OR, NOT, etc.).
- Used in arithmetic circuits.
- Used in communication systems.
- Used in embedded systems and VLSI design.

Small Gate  
Big Possibilities! 😊

“NOR Gate – True Only When There is No Input!” 😊

Different Inputs  
Different Output ! 😊

# EX-OR (XOR) & EX-NOR (XNOR) GATES

Definition, Symbols, Truth Tables, Working and Applications

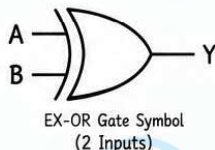
Same Inputs  
Same Output ! 😊

## 1. EX-OR GATE (XOR)

### 1. Definition

- An EX-OR gate is a basic logic gate that gives an output of 1 (HIGH) when the inputs are **different**.
- The output is 0 (LOW) when the inputs are the same.
- It is also called an **Exclusive-OR gate**.
- It has two or more inputs and one output.

### 2. Symbol



### 3 Boolean Expression

$$Y = A \oplus B$$

Where,

- A = Input 1
- B = Input 2
- Y = Output
- $\oplus$  denotes EX-OR operation (exclusive OR)

### 4. Truth Table

| A<br>(Input 1) | B<br>(Input 2) | Y = A $\oplus$ B<br>(Output) |
|----------------|----------------|------------------------------|
| 0              | 0              | 0                            |
| 0              | 1              | 1                            |
| 1              | 0              | 1                            |
| 1              | 1              | 0                            |

### 5. Working

- Compares the two inputs.
- If the inputs are different (0,1 or 1,0) → Output is 1.
- If the inputs are the same (0,0 or 1,1) → Output is 0.

### 6. Example

A = 0, B = 1 → Y = 1  
(inputs are different)

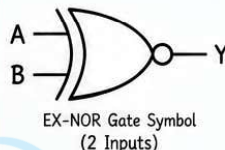
A = 1, B = 1 → Y = 0  
(inputs are same)

## 2. EX-NOR GATE (XNOR)

### 1. Definition

- An EX-NOR gate is a basic logic gate that gives an output of 1 (HIGH) when the inputs are the **same**.
- The output is 0 (LOW) when the inputs are different.
- It is also called an **Exclusive-NOR gate** or **Equivalence gate**.
- It has two or more inputs and one output.

### 2. Symbol



### 3 Boolean Expression

$$Y = A \ominus B$$

Where,

- A = Input 1
- B = Input 2
- Y = Output
- $\ominus$  denotes EX-NOR operation (equivalence)

### 4. Truth Table

| A<br>(Input 1) | B<br>(Input 2) | Y = A $\ominus$ B<br>(Output) |
|----------------|----------------|-------------------------------|
| 0              | 0              | 1                             |
| 0              | 1              | 0                             |
| 1              | 0              | 0                             |
| 1              | 1              | 1                             |

### 6. Example

A = 0, B = 0 → Y = 1  
(inputs are same)

A = 1, B = 0 → Y = 0  
(inputs are different)

### 5. Working

- Compares the two inputs.
- If the inputs are the same (0,0 or 1,1) → Output is 1.
- If the inputs are different (0,1 or 1,0) → Output is 0.

### 7. Applications

- Used in parity checkers.
- Used in data comparison (equality detection).
- Used in error detection and correction.
- Used in digital communication systems.
- Used in control systems.
- Used in memory and data storage circuits.

Learn  
Practice  
Build  
Succeed



A small gate...  
A big possibility ! 😊

# UNIVERSAL GATES

## NAND and NOR – The Universal Logic Gates

With only NAND  
or NOR, we can  
build any logic circuit ! 😊

### 1. What are Universal Gates ?

- A universal gate is a logic gate that can be used to implement any other logic gate (NOT, AND, OR, etc.).
- NAND and NOR gates** are called **universal gates**.
- Using only NAND gates or only NOR gates, we can design any digital logic circuit.
- This makes them very useful in digital electronics and integrated circuit design.

### 2. NAND Gate (Universal Gate)

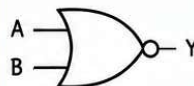


$$Y = \overline{A \cdot B}$$

(NOT-AND)

| Truth Table (NAND) |   |   |
|--------------------|---|---|
| A                  | B | Y |
| 0                  | 0 | 1 |
| 0                  | 1 | 1 |
| 1                  | 0 | 1 |
| 1                  | 1 | 0 |

### 3. NOR Gate (Universal Gate)



$$Y = \overline{A + B}$$

(NOT-OR)

| Truth Table (NOR) |   |   |
|-------------------|---|---|
| A                 | B | Y |
| 0                 | 0 | 1 |
| 0                 | 1 | 0 |
| 1                 | 0 | 0 |
| 1                 | 1 | 0 |

### 4. Why are NAND and NOR Universal ?

- Because they are **functionally complete**.
- They can be used to implement NOT, AND, OR and any other gate.
- Any Boolean function can be realized using only NAND gates or only NOR gates.
- Hence, they are called **universal gates**.

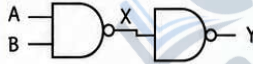
### 5. Implementing Basic Gates using NAND (Only NAND)

#### (a) NOT Gate using NAND



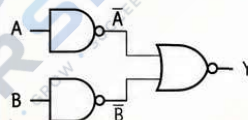
$$Y = \overline{A \cdot A} = \overline{A}$$

#### (b) AND Gate using NAND



$$Y = \overline{X \cdot X} = \overline{(\overline{A \cdot B})} = A \cdot B$$

#### (c) OR Gate using NAND



$$Y = \overline{\overline{A} \cdot \overline{B}} = A + B$$

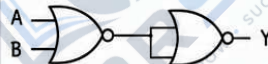
### 6. Implementing Basic Gates using NOR (Only NOR)

#### (a) NOT Gate using NOR



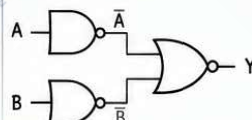
$$Y = \overline{A + A} = \overline{A}$$

#### (b) OR Gate using NOR



$$Y = \overline{(\overline{A + B})} = A + B$$

#### (c) AND Gate using NOR



$$Y = \overline{\overline{A} + \overline{B}} = A \cdot B$$

### 7. Key Points

- ✓ NAND and NOR are universal gates.
- ✓ They can be used to implement all other logic gates.
- ✓ Any Boolean expression can be realized using only NAND or only NOR.
- ✓ NAND is NOT-AND, NOR is NOT-OR.
- ✓ NAND and NOR simplify hardware design.
- ✓ They are widely used in digital ICs and programmable logic devices (PLDs).

### 8. Applications

- Used in designing complex digital circuits.
- Used in programmable logic devices (CPLD, FPGA).
- Used in memory circuits.
- Used in microprocessors and controllers.
- Useful in VLSI design (fewer gate types needed).
- Helps in reducing the number of different components.
- Used in building other gates and logic functions.

### 9. Conclusion

- NAND and NOR gates are called universal gates because they can implement all other logic gates.
- They are essential building blocks in digital electronics.
- Using only one type of universal gate, we can design any logical function or complete digital system.

Learn  
Practice  
Build  
Succeed



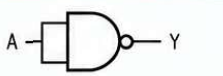
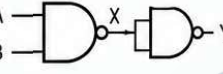
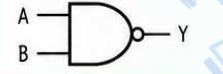
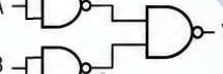
Different Design Same Logic ! 😊

# Designing of Basic Gates using NAND and NOR Gates

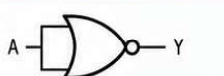
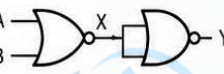
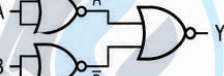
Any logic gate can be implemented using only NAND gates or only NOR gates (Universal Gates)

NAND or NOR You can build Everything ! 😊

## 1. Using NAND Gates (Only NAND)

| Logic Gate          | Circuit using NAND gates                                                          | Boolean Expression                | Working / Explanation                                                        |
|---------------------|-----------------------------------------------------------------------------------|-----------------------------------|------------------------------------------------------------------------------|
| NOT Gate (Inverter) |  | $Y = \bar{A} = A \text{ NAND } A$ | Both inputs are same, so output is complement of A.                          |
| AND Gate            |  | $Y = (A \cdot B)$                 | First NAND gives $(A \cdot B)$ , second NAND inverts it to get $A \cdot B$ . |
| OR Gate             |  | $Y = A + B$                       | Using De Morgan's theorem:<br>$A + B = (\bar{A} \cdot \bar{B})$              |
| NAND Gate (Direct)  |  | $Y = (A \cdot \bar{B})$           | Already a NAND gate.                                                         |
| NOR Gate            |  | $Y = \overline{(A + B)}$          | Using De Morgan's theorem:<br>$(A + B) = \bar{A} \cdot \bar{B}$              |

## 2. Using NOR Gates (Only NOR)

| Logic Gate          | Circuit using NOR gates                                                            | Boolean Expression               | Working / Explanation                                                 |
|---------------------|------------------------------------------------------------------------------------|----------------------------------|-----------------------------------------------------------------------|
| NOT Gate (Inverter) |  | $Y = \bar{A} = A \text{ NOR } A$ | Both inputs are same, so output is complement of A.                   |
| OR Gate             |  | $Y = A + B$                      | First NOR gives $(A + B)$ , second NOR inverts it to get $A + B$ .    |
| AND Gate            |  | $Y = A \cdot B$                  | Using De Morgan's theorem:<br>$A \cdot B = (\bar{A} + \bar{B})$       |
| NOR Gate (Direct)   |  | $Y = \overline{(A + B)}$         | Already a NOR gate.                                                   |
| NAND Gate           |  | $Y = (A \cdot \bar{B})$          | Using De Morgan's theorem:<br>$(A \cdot \bar{B}) = \bar{A} + \bar{B}$ |

## 3. Key Points

1. NAND and NOR gates are universal gates.
2. Any logic gate (NOT, AND, OR, NAND, NOR) can be implemented using only NAND gates or only NOR gates.
3. NAND is NOT-AND and NOR is NOT-OR.
4. We use De Morgan's theorems to convert expressions.
5. Useful in digital IC design when limited gate types are available.

## 4. De Morgan's Theorems

| Law                      | Expression                                   |
|--------------------------|----------------------------------------------|
| 1. Complement of product | $\overline{(A \cdot B)} = \bar{A} + \bar{B}$ |
| 2. Complement of sum     | $\overline{(A + B)} = \bar{A} \cdot \bar{B}$ |

## 5. Conclusion

- ✓ Using only NAND gates or only NOR gates, we can design all basic logic gates.
- ✓ This property makes them very important in digital electronics and VLSI design.
- ✓ NAND and NOR gates reduce the number of different components required.
- ✓ They provide flexibility, cost-effectiveness and ease of implementation.

Learn Practice Build Succeed



Digital Electronics

"Universal Gates – Small in Number, Infinite in Possibilities !"

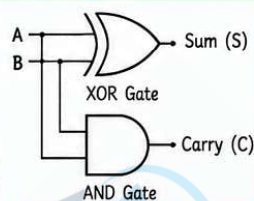


## 1. Half Adder

### 1. Definition

- A half adder is a combinational logic circuit that adds **two 1-bit binary inputs** (A and B).
- It produces two outputs:
  - Sum (S)
  - Carry (C)
- It does not consider any carry input (there is **no input carry**).
- It is the simplest form of binary adder.

### 2. Logic Diagram



### 3. Boolean Expressions

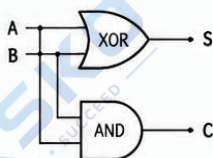
$$\text{Sum (S)} = A \oplus B$$

$$\text{Carry (C)} = A \cdot B$$

### 4. Truth Table

| A | B | S (Sum) | C (Carry) |
|---|---|---------|-----------|
| 0 | 0 | 0       | 0         |
| 0 | 1 | 1       | 0         |
| 1 | 0 | 1       | 0         |
| 1 | 1 | 0       | 1         |

### 5. Half Adder Circuit (Complete)

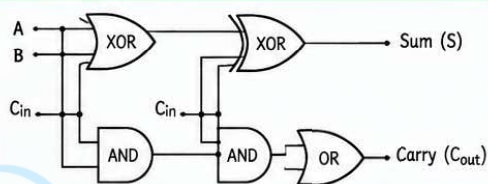


## 2. Full Adder

### 1. Definition

- A full adder is a combinational logic circuit that adds **three 1-bit binary inputs** (A, B and Cin).
- It produces two outputs:
  - Sum (S)
  - Carry (C<sub>out</sub>)
- It takes into account the **carry input** from the previous lower significant bit.
- It is widely used in multi-bit binary addition (e.g., in adders, ALU of processors).

### 2. Logic Diagram



### 3. Boolean Expressions

$$\text{Sum (S)} = A \oplus B \oplus C_{in}$$

$$\text{Carry (C}_{out}\text{)} = A \cdot B + (A \oplus B) \cdot C_{in}$$

### 4. Truth Table

| A | B | Cin | S (Sum) | Cout (Carry) |
|---|---|-----|---------|--------------|
| 0 | 0 | 0   | 0       | 0            |
| 0 | 0 | 1   | 1       | 0            |
| 0 | 1 | 0   | 1       | 0            |
| 0 | 1 | 1   | 0       | 1            |
| 1 | 0 | 0   | 1       | 0            |
| 1 | 0 | 1   | 0       | 1            |
| 1 | 1 | 0   | 0       | 1            |
| 1 | 1 | 1   | 1       | 1            |

## 5. Key Differences

| Feature                    | Half Adder              | Full Adder                                    |
|----------------------------|-------------------------|-----------------------------------------------|
| Inputs                     | 2 (A, B)                | 3 (A, B, Cin)                                 |
| Outputs                    | 2 (S, C)                | 2 (S, C <sub>out</sub> )                      |
| Carry Input                | Not considered          | Considered (Cin)                              |
| Boolean Expression (Sum)   | $A \oplus B$            | $A \oplus B \oplus C_{in}$                    |
| Boolean Expression (Carry) | $A \cdot B$             | $A \cdot B + (A \oplus B) \cdot C_{in}$       |
| Usage                      | Simple addition (1 bit) | Used in multi-bit adders (e.g., ripple adder) |

## 6. Applications

- Used in arithmetic and logic units (ALU).
- Used in calculators and microprocessors.
- Used in digital computers for binary addition.
- Used in data processing and signal processing.
- Half adders are building blocks for full adders.
- Full adders are used to design multi-bit adders (e.g., ripple carry adder, carry look-ahead adder).
- Used in embedded systems and VLSI design.

"Adders may be small, but they are the backbone of all calculations !" 😊

## 7. Key Points

- ✓ Half adder adds two bits, full adder adds three bits.
- ✓ Half adder does not have carry input, full adder has carry input.
- ✓ Sum is obtained using XOR operation.
- ✓ Carry is obtained using AND and OR operations.
- ✓ Full adder can be built using two half adders and an OR gate.
- ✓ Used in digital systems for performing binary addition.
- ✓ Essential part of CPUs, ALUs and other arithmetic circuits.



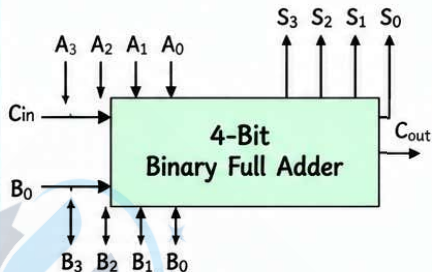
# 4-Bit Binary Full Adder

Adds two 4-bit binary numbers and gives a 4-bit sum with a carry output

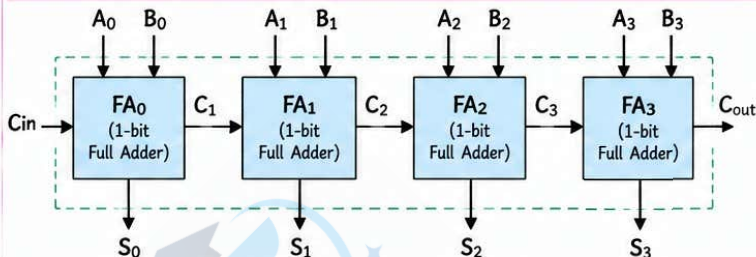
## 1. Introduction

- A 4-bit binary full adder adds two 4-bit binary numbers ( $A_3 A_2 A_1 A_0$ ) and ( $B_3 B_2 B_1 B_0$ ) with an optional carry input ( $C_{in}$ ).
- It produces a 4-bit sum ( $S_3 S_2 S_1 S_0$ ) and a carry output ( $C_{out}$ ).
- It is built by **cascading four 1-bit full adders**.
- Each full adder adds one bit along with a carry from the previous stage.

## 2. Block Diagram



## 3. Logic Diagram (Using Four 1-bit Full Adders)



Each full adder adds one bit and the carry from the previous stage.

## 4. Truth Table (1-bit Full Adder)

| A | B | Cin | Sum (S) | Cout |
|---|---|-----|---------|------|
| 0 | 0 | 0   | 0       | 0    |
| 0 | 0 | 1   | 1       | 0    |
| 0 | 1 | 0   | 1       | 0    |
| 0 | 1 | 1   | 0       | 1    |
| 1 | 0 | 0   | 1       | 0    |
| 1 | 0 | 1   | 0       | 1    |
| 1 | 1 | 0   | 0       | 1    |
| 1 | 1 | 1   | 1       | 1    |

## 5. Boolean Expressions (1-bit Full Adder)

$$\text{Sum (S)} = A \oplus B \oplus C_{in}$$

$$\text{Carry (C}_{out}\text{)} = AB + BC_{in} + AC_{in}$$

Alternatively,

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

For 4-bit adder (bit-wise):

$$S_0 = A_0 \oplus B_0 \oplus C_{in}$$

$$C_1 = A_0 B_0 + B_0 C_{in} + A_0 C_{in}$$

$$S_1 = A_1 \oplus B_1 \oplus C_1$$

$$C_2 = A_1 B_1 + B_1 C_1 + A_1 C_1$$

$$S_2 = A_2 \oplus B_2 \oplus C_2$$

$$C_3 = A_2 B_2 + B_2 C_2 + A_2 C_2$$

$$S_3 = A_3 \oplus B_3 \oplus C_3$$

$$C_{out} = A_3 B_3 + B_3 C_3 + A_3 C_3$$

## 6. Example (4-bit Addition)

Add the two 4-bit numbers:

$$A = 1101_2 \text{ (13)}$$

$$B = 0111_2 \text{ (7)}$$

$$C_{in} = 0$$

| Bit     | A | B | Cin | Sum | Cout |
|---------|---|---|-----|-----|------|
| 0 (LSB) | 1 | 1 | 0   | 0   | 1    |
| 1       | 0 | 1 | 1   | 0   | 1    |
| 2       | 1 | 1 | 1   | 1   | 1    |
| 3 (MSB) | 1 | 0 | 1   | 0   | 1    |

Result:

$$A + B = 1101_2 + 0111_2 = 1\ 0100_2 = 20_{10}$$

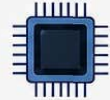
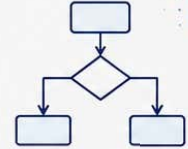
## 7. Key Points

- ✓ A 4-bit full adder uses **four 1-bit full adders** in **cascade**.
- ✓ Each stage adds one bit and the carry from the previous stage.
- ✓ It has **two 4-bit inputs** (A and B), one **carry input** ( $C_{in}$ ), 4-bit sum output and **one carry output** ( $C_{out}$ ).
- ✓ Widely used in ALU of microprocessors.
- ✓ Can be extended to **n-bits** by cascading more full adders.
- ✓ Useful in arithmetic and logic operations in digital systems.

## 8. Applications

- Used in Arithmetic Logic Unit (ALU).
- Used in microprocessors and CPUs.
- Used in calculators and digital computers.
- Used in signal processing and data processing.
- Can be used in larger adders (8-bit, 16-bit, etc.).

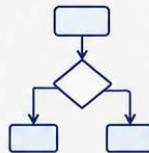
1010101010  
0101010101  
1010101010  
0101010101



# Unit - III



10110  
01001  
10101



Logic is the language of Computers ! 😊

# Boolean Algebra

Fundamentals, Laws, Theorems and Simplification

Simple Rules  
Powerful Circuits  
Better Future ! 😊

## 1. What is Boolean Algebra ?

- Boolean Algebra is a mathematical system for dealing with **binary variables** (0 and 1) and logical operations.
- It was developed by **George Boole** in 1854.
- It is the foundation of **digital electronics** and computer systems.
- It uses logic operations like **AND** ( $\cdot$ ), **OR** ( $+$ ) and **NOT** ( $\bar{\phantom{x}}$ ).
- Variables can take only two values: 0 (False) or 1 (True).

## 2. Basic Operators and Truth Tables

| Operation                              | Symbol                                                                            | Boolean Expression | Truth Table |   |   |
|----------------------------------------|-----------------------------------------------------------------------------------|--------------------|-------------|---|---|
| <b>AND</b><br>(Logical Multiplication) |  | $Y = A \cdot B$    | A           | B | Y |
|                                        |                                                                                   |                    | 0           | 0 | 0 |
|                                        |                                                                                   |                    | 0           | 1 | 0 |
|                                        |                                                                                   |                    | 1           | 0 | 0 |
|                                        |                                                                                   |                    | 1           | 1 | 1 |
| <b>OR</b><br>(Logical Addition)        |  | $Y = A + B$        | A           | B | Y |
|                                        |                                                                                   |                    | 0           | 0 | 0 |
|                                        |                                                                                   |                    | 0           | 1 | 1 |
|                                        |                                                                                   |                    | 1           | 0 | 1 |
|                                        |                                                                                   |                    | 1           | 1 | 1 |
| <b>NOT</b><br>(Complement)             |  | $Y = \bar{A}$      | A           | Y |   |
|                                        |                                                                                   |                    | 0           | 1 |   |
|                                        |                                                                                   |                    | 1           | 0 |   |

## 3. Basic Laws of Boolean Algebra

| Law              | Expression                                                                             |
|------------------|----------------------------------------------------------------------------------------|
| Commutative Law  | $A + B = B + A$<br>$A \cdot B = B \cdot A$                                             |
| Associative Law  | $(A + B) + C = A + (B + C)$<br>$(A \cdot B) \cdot C = A \cdot (B \cdot C)$             |
| Distributive Law | $A \cdot (B + C) = A \cdot B + A \cdot C$<br>$A + (B \cdot C) = (A + B) \cdot (A + C)$ |
| Identity Law     | $A + 0 = A$<br>$A \cdot 1 = A$                                                         |
| Null Law         | $A + 1 = 1$<br>$A \cdot 0 = 0$                                                         |
| Idempotent Law   | $A + A = A$                                                                            |
| Complement Law   | $A + \bar{A} = 1$<br>$A \cdot \bar{A} = 0$                                             |
| Involution Law   | $\bar{\bar{A}} = A$                                                                    |

## 4. Derived Laws and Theorems

| Law / Theorem                | Expression                                                                               |
|------------------------------|------------------------------------------------------------------------------------------|
| Absorption Law               | $A + A \cdot B = A$<br>$A \cdot (A + B) = A$                                             |
| De Morgan's Theorem          | $\overline{A + B} = \bar{A} \cdot \bar{B}$<br>$\overline{A \cdot B} = \bar{A} + \bar{B}$ |
| Consensus Theorem            | $A \cdot B + \bar{A} \cdot C + B \cdot C = A \cdot B + \bar{A} \cdot C$                  |
| Duality Principle            | Interchange $+$ and $\cdot$<br>Interchange 0 and 1<br>to get the dual expression.        |
| Principle of Complementation | If $F = 0$ , then $\bar{F} = 1$<br>If $F = 1$ , then $\bar{F} = 0$                       |

Duality: Every Boolean expression has a dual obtained by interchanging  $+$  and  $\cdot$  and 0 and 1.

## 5. Important Boolean Identities

- $A + 0 = A$
- $A + 1 = 1$
- $A \cdot 0 = 0$
- $A \cdot 1 = A$
- $A + A = A$
- $A \cdot A = A$
- $A + \bar{A} = 1$
- $A \cdot \bar{A} = 0$
- $\bar{\bar{A}} = A$
- $A + A \cdot B = A$
- $A \cdot (A + B) = A$

## 6. De Morgan's Theorem (Proof using Truth Table)

| A | B | A + B | $\overline{A + B}$ | A · B | $\overline{A \cdot B}$ |
|---|---|-------|--------------------|-------|------------------------|
| 0 | 0 | 0     | 1                  | 0     | 1                      |
| 0 | 1 | 1     | 0                  | 0     | 1                      |
| 1 | 0 | 1     | 0                  | 0     | 1                      |
| 1 | 1 | 1     | 0                  | 1     | 0                      |

Hence,

- $\overline{A + B} = \bar{A} \cdot \bar{B}$
- $\overline{A \cdot B} = \bar{A} + \bar{B}$

## 7. Simplification Example

Simplify the expression:  $F = A \cdot (A + B) + \bar{A} \cdot B$

**Solution:**

$$\begin{aligned}
 F &= A \cdot (A + B) + \bar{A} \cdot B \\
 &= A \cdot A + A \cdot B + \bar{A} \cdot B \\
 &= A + A \cdot B + \bar{A} \cdot B \quad (\text{since } A \cdot A = A) \\
 &= A + B \cdot (A + \bar{A}) \quad (\text{taking } B \text{ common}) \\
 &= A + B \cdot 1 \\
 &= A + B
 \end{aligned}$$

Thus,  $F = A + B$

## 8. Applications of Boolean Algebra

- Simplification of logic circuits.
- Design of combinational and sequential circuits.
- Minimization of logic expressions (using K-maps, etc.).
- Used in computer arithmetic and control systems.
- Design of memory circuits and digital devices.
- Essential in VLSI design and logic optimization.

## 9. Key Takeaways

- ✓ Boolean Algebra deals with binary logic (0 and 1).
- ✓ It follows a set of laws and theorems.
- ✓ Helps in simplifying and implementing logic circuits.
- ✓ Widely used in digital electronics, computers and communication systems.

Learn  
Practice  
Simplify  
Succeed



Digital Electronics

" Boolean Algebra Turns Logic into Possibilities ! "



Same Laws  
Many Possibilities ! 😊

# Boolean Laws

Fundamental Rules of Boolean Algebra for Logical Simplification

Simplify Logic  
Design Better  
Think Boolean ! 😊

## 1. Commutative Laws

The order of variables does not change the result.

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

Useful when rearranging variables.

## 2. Associative Laws

The grouping of variables does not change the result.

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

Useful when dealing with multiple variables.

## 3. Distributive Laws

AND distributes over OR, and OR distributes over AND.

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Very useful in simplification and circuit design.

## 4. Identity Laws

0 and 1 act as identity elements.

$$A + 0 = A$$

$$A \cdot 1 = A$$

Adding 0 or multiplying by 1 does not change the value.

## 5. Null (Domination) Laws

0 and 1 dominate in OR and AND respectively.

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

OR with 1 gives 1.  
AND with 0 gives 0.

## 6. Idempotent Laws

A variable ORed or ANDed with itself gives the same variable.

$$A + A = A$$

$$A \cdot A = A$$

Useful in simplifying expressions.

## 7. Complement Laws

A variable and its complement give fixed results.

$$A + \bar{A} = 1$$

$$A \cdot \bar{A} = 0$$

A variable and its complement are always opposite.

## 8. Involution Law

The complement of a complement is the original variable.

$$\bar{\bar{A}} = A$$

Taking complement twice brings back the original value.

## 9. De Morgan's Laws

The complement of a sum is the product of the complements, and the complement of a product is the sum of the complements.

$$\overline{(A + B)} = \bar{A} \cdot \bar{B}$$

$$\overline{(A \cdot B)} = \bar{A} + \bar{B}$$

Very important in digital circuit design and simplification.

## 10. Absorption Laws

A variable absorbs the term containing itself.

$$A + A \cdot B = A$$

$$A \cdot (A + B) = A$$

Helps in reducing complex expressions.

## 11. Consensus Theorem

Eliminates redundant terms.

$$A \cdot B + \bar{A} \cdot C + B \cdot C = A \cdot B + \bar{A} \cdot C$$

Useful in simplifying three-variable expressions.

## 12. Duality Principle

- Every Boolean expression has a dual.
- To get the dual, interchange + and · and also interchange 0 and 1.
- If an expression is valid, its dual is also valid.
- Example: Dual of  $(A + 0) = A$  is  $(A \cdot 1) = A$ .

Learn  
Practice  
Simplify  
Succeed

"Boolean Laws Make Complex Logic Simple !"



Digital Electronics

Same  
Output  
Different Form ! 😊

# De-Morgan's Theorem

A Useful Tool to Simplify and Implement Logic Circuits !

De-Morgan's Theorem  
Helps in Logic  
Simplification ! 😊

## 1. Introduction

- De-Morgan's Theorems are two important laws in Boolean algebra.
- They relate the complement of a sum (OR) and the complement of a product (AND).
- They are very useful for simplifying logic expressions and in designing logic circuits using **only NAND or only NOR gates**.
- These theorems help in converting complex expressions into simpler equivalent forms.

## 2. De-Morgan's Theorems

### First Theorem

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

"The complement of a sum is equal to the product of the complements."

### Second Theorem

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

"The complement of a product is equal to the sum of the complements."

## 3. Proof using Truth Tables

First Theorem :  $\overline{A + B} = \bar{A} \cdot \bar{B}$

| A | B | A + B | $\overline{A + B}$ | $\bar{A}$ | $\bar{B}$ | $\bar{A} \cdot \bar{B}$ |
|---|---|-------|--------------------|-----------|-----------|-------------------------|
| 0 | 0 | 0     | 1                  | 1         | 1         | 1                       |
| 0 | 1 | 1     | 0                  | 1         | 0         | 0                       |
| 1 | 0 | 1     | 0                  | 0         | 1         | 0                       |
| 1 | 1 | 1     | 0                  | 0         | 0         | 0                       |

$$\overline{A + B} = \bar{A} \cdot \bar{B} \quad (\text{LHS} = \text{RHS})$$

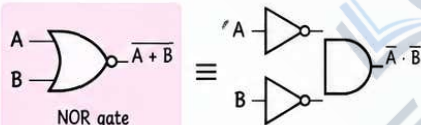
Second Theorem :  $\overline{A \cdot B} = \bar{A} + \bar{B}$

| A | B | A · B | $\overline{A \cdot B}$ | $\bar{A}$ | $\bar{B}$ | $\bar{A} + \bar{B}$ |
|---|---|-------|------------------------|-----------|-----------|---------------------|
| 0 | 0 | 0     | 1                      | 1         | 1         | 1                   |
| 0 | 1 | 0     | 1                      | 1         | 0         | 1                   |
| 1 | 0 | 0     | 1                      | 0         | 1         | 1                   |
| 1 | 1 | 1     | 0                      | 0         | 0         | 0                   |

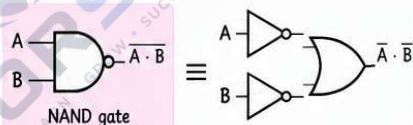
$$\overline{A \cdot B} = \bar{A} + \bar{B} \quad (\text{LHS} = \text{RHS})$$

## 4. Logic Gate Representation

First Theorem :  $\overline{A + B} = \bar{A} \cdot \bar{B}$



Second Theorem :  $\overline{A \cdot B} = \bar{A} + \bar{B}$



## 5. Examples

Example 1 : Simplify  $\overline{(A + B + C)}$

Using De-Morgan's theorem:

$$\overline{A + B + C} = \bar{A} \cdot \bar{B} \cdot \bar{C}$$

Complement of a sum becomes product of complements.

Example 2 : Simplify  $\overline{(A \cdot B \cdot C)}$

Using De-Morgan's theorem:

$$\overline{A \cdot B \cdot C} = \bar{A} + \bar{B} + \bar{C}$$

Complement of a product becomes sum of complements.

## 6. Applications

- Useful for simplifying Boolean expressions.
- Helps in converting between NAND and NOR implementations.
- Widely used in digital circuit design and logic minimization.
- Helps in realizing any logic function using only NAND gates or only NOR gates.
- Important in VLSI design and computer architecture.

## 7. Key Points

- ✓ De-Morgan's Theorems provide equivalent expressions.
- ✓ Complement of OR  $\rightarrow$  AND of complements.
- ✓ Complement of AND  $\rightarrow$  OR of complements.
- ✓ Verified using truth tables.
- ✓ Very useful for logic simplification and circuit design.
- ✓ Used to implement logic circuits using only NAND or only NOR.

## 8. Conclusion

- De-Morgan's Theorems are fundamental laws in Boolean algebra.
- They establish a relationship between AND, OR and complement operations.
- They simplify logic expressions and are essential in digital electronics design.

"Different Form, Same Result !" 😊

Learn  
Practice  
Simplify  
Succeed

Digital Electronics

Simplify  
Logic  
Simplify Life! 😊

# Simplification of Boolean Equations using Boolean Algebra

Step-by-Step Methods • Solved Examples • Easy to Understand

Same Logic  
Many Steps  
One Simple Result! 😊

## 1. Why Simplify?

Simplification of Boolean equations helps to:

- Reduce the number of logic gates.
- Minimize the cost and power consumption.
- Improve the speed and performance of digital circuits.
- Make the circuit easy to implement and understand.

## 2. Basic Boolean Identities Used

| Law / Identity   | Expression                                                                                |                         |
|------------------|-------------------------------------------------------------------------------------------|-------------------------|
| Identity Law     | $A + 0 = A$                                                                               | $A \cdot 1 = A$         |
| Null Law         | $A + 1 = 1$                                                                               | $A \cdot 0 = 0$         |
| Idempotent Law   | $A + A = A$                                                                               | $A \cdot A = A$         |
| Complement Law   | $A + \bar{A} = 1$                                                                         | $A \cdot \bar{A} = 0$   |
| Commutative Law  | $A + B = B + A$                                                                           | $A \cdot B = B \cdot A$ |
| Associative Law  | $(A + B) + C = A + (B + C)$<br>$(A \cdot B) \cdot C = A \cdot (B \cdot C)$                |                         |
| Distributive Law | $A \cdot (B + C) = A \cdot B + A \cdot C$<br>$A + (B \cdot C) = (A + B) \cdot (A + C)$    |                         |
| De Morgan's Law  | $\overline{(A + B)} = \bar{A} \cdot \bar{B}$ $\overline{(A \cdot B)} = \bar{A} + \bar{B}$ |                         |

## 3. Steps for Simplification

- 1 Remove parentheses (if possible) using Boolean laws.
- 2 Apply known Boolean identities.
- 3 Combine like terms.
- 4 Eliminate redundant terms.
- 5 Arrive at the simplest form.

"Use Boolean laws step by step, not randomly." 😊

## 4. Example 1

Simplify :  $F = A + A \cdot B$

**Solution :**

$$\begin{aligned} F &= A + A \cdot B \\ &= A \cdot (1 + B) \quad [\text{Distributive Law}] \\ &= A \cdot 1 \quad [\text{Identity Law}] \\ &= A \end{aligned}$$

$$\therefore F = A$$

## 5. Example 2

Simplify :  $F = (A + B) \cdot (A + \bar{B})$

**Solution :**

$$\begin{aligned} F &= (A + B) \cdot (A + \bar{B}) \\ &= A + B \cdot \bar{B} \quad [\text{Distributive Law}] \\ &= A + 0 \quad [\text{Complement Law}] \\ &= A \end{aligned}$$

$$\therefore F = A$$

## 6. Example 3

Simplify :  $F = A \cdot B + A \cdot \bar{B}$

**Solution :**

$$\begin{aligned} F &= A \cdot B + A \cdot \bar{B} \\ &= A \cdot (B + \bar{B}) \quad [\text{Distributive Law}] \\ &= A \cdot 1 \quad [\text{Complement Law}] \\ &= A \end{aligned}$$

$$\therefore F = A$$

## 7. Example 4

Simplify :  $F = \bar{A} \cdot (A + B)$

**Solution :**

$$\begin{aligned} F &= \bar{A} \cdot (A + B) \\ &= \bar{A} \cdot A + \bar{A} \cdot B \quad [\text{Distributive Law}] \\ &= 0 + \bar{A} \cdot B \quad [\text{Complement Law}] \\ &= \bar{A} \cdot B \end{aligned}$$

$$\therefore F = \bar{A} \cdot B$$

## 8. Example 5

Simplify :  $F = (A + B) (\bar{A} + \bar{B})$

**Solution :**

$$\begin{aligned} F &= (A + B) (\bar{A} + \bar{B}) \\ &= A \cdot \bar{A} + A \cdot \bar{B} + B \cdot \bar{A} + B \cdot \bar{B} \\ &= 0 + A \cdot \bar{B} + \bar{A} \cdot B + 0 \\ &= A \cdot \bar{B} + \bar{A} \cdot B \\ &= B(A + \bar{A}) + \bar{B}(\bar{A} + A) \quad [\text{Distributive Law}] \\ &= B \cdot 1 + \bar{B} \cdot 1 \quad [\text{Complement Law}] \\ &= B + \bar{B} \\ &= 1 \end{aligned}$$

$$\therefore F = B$$

## 9. Example 6 (Using Multiple Laws)

Simplify :  $F = A + \bar{A} \cdot B + A \cdot B$

**Solution :**

$$\begin{aligned} F &= A + \bar{A} \cdot B + A \cdot B \\ &= A + B(\bar{A} + A) \quad [\text{Distributive Law}] \\ &= A + B \cdot 1 \quad [\text{Complement Law}] \\ &= A + B \end{aligned}$$

$$\therefore F = A + B$$

## 10. Example 7 (Redundant Terms)

Simplify :  $F = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B$

**Solution :**

$$\begin{aligned} F &= A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B \\ &= A(B + \bar{B}) + \bar{A} \cdot B \quad [\text{Distributive Law}] \\ &= A \cdot 1 + \bar{A} \cdot B \quad [\text{Complement Law}] \\ &= A + \bar{A} \cdot B \end{aligned}$$

$$\therefore F = A + \bar{A} \cdot B$$

## 11. Practice Questions

Simplify the following :

- 1  $A + A \cdot B$
- 2  $(A + B) \cdot (A + \bar{B})$
- 3  $A \cdot (\bar{A} + B)$
- 4  $A + \bar{A} \cdot B + B$
- 5  $(A \cdot B) + (A \cdot \bar{B})$

## 12. Key Takeaways

- ✓ Use Boolean laws step by step.
- ✓ Look for common terms and apply distributive law.
- ✓ Remove redundant terms.
- ✓ Simplify to the minimum number of literals.
- ✓ Simplified expressions lead to simpler, faster and more efficient digital circuits.

Learn  
Practice  
Simplify  
Succeed



Digital Electronics

"Simpler Expressions, Smarter Circuits!" 😊

Every Boolean Function can be written as a Sum of Products ! 😊

# Fundamental Products & Sum Terms

## Building Blocks of Boolean Expressions !

Understand Products and Sums Simplify Digital Logic ! 😊

### 1. What are Products and Sums ?

- In Boolean algebra, a **product term** is formed by ANDing (·) one or more literals (variables or their complements).
- A sum term is formed by ORing (+) one or more literals.
- These are called fundamental terms because any Boolean function can be expressed as a sum of products (**SOP**) or a product of sums (**POS**).

### 2. Literals, Product and Sum Terms

| Term                        | Symbol       | Example                   | Meaning                      |
|-----------------------------|--------------|---------------------------|------------------------------|
| Literal                     | A, $\bar{A}$ | A, $\bar{B}$ , C          | A variable or its complement |
| Product term (Minterm form) | · (AND)      | $A \cdot B \cdot \bar{C}$ | AND of literals              |
| Sum term (Maxterm form)     | + (OR)       | $A + \bar{B} + C$         | OR of literals               |

### 3. Minterm and Maxterm

- A **minterm** is a product term that contains all variables of the function (each variable appears either in true or complemented form).
- A **maxterm** is a sum term that contains all variables of the function (each variable appears either in true or complemented form).
- For **n** variables, there are  **$2^n$  minterms** and  **$2^n$  maxterms**.

### 4. Minterms for 3 Variables (A, B, C)

| Decimal equivalent | A | B | C | Minterm                       |
|--------------------|---|---|---|-------------------------------|
| 0                  | 0 | 0 | 0 | $m_0 = \bar{A}\bar{B}\bar{C}$ |
| 1                  | 0 | 0 | 1 | $m_1 = \bar{A}\bar{B}C$       |
| 2                  | 0 | 1 | 0 | $m_2 = \bar{A}B\bar{C}$       |
| 3                  | 0 | 1 | 1 | $m_3 = \bar{A}BC$             |
| 4                  | 1 | 0 | 0 | $m_4 = A\bar{B}\bar{C}$       |
| 5                  | 1 | 0 | 1 | $m_5 = A\bar{B}C$             |
| 6                  | 1 | 1 | 0 | $m_6 = AB\bar{C}$             |
| 7                  | 1 | 1 | 1 | $m_7 = ABC$                   |

### 5. Maxterms for 3 Variables (A, B, C)

| Decimal equivalent | A | B | C | Maxterm                             |
|--------------------|---|---|---|-------------------------------------|
| 0                  | 0 | 0 | 0 | $M_0 = A + B + C$                   |
| 1                  | 0 | 0 | 1 | $M_1 = A + B + \bar{C}$             |
| 2                  | 0 | 1 | 0 | $M_2 = A + \bar{B} + C$             |
| 3                  | 0 | 1 | 1 | $M_3 = A + \bar{B} + \bar{C}$       |
| 4                  | 1 | 0 | 0 | $M_4 = \bar{A} + B + C$             |
| 5                  | 1 | 0 | 1 | $M_5 = \bar{A} + B + \bar{C}$       |
| 6                  | 1 | 1 | 0 | $M_6 = \bar{A} + \bar{B} + C$       |
| 7                  | 1 | 1 | 1 | $M_7 = \bar{A} + \bar{B} + \bar{C}$ |

### 6. Canonical Forms

#### Sum of Products (SOP) form

The function is written as a sum (OR) of minterms for which the output is 1.  
 $F(A, B, C) = \sum m(\text{list of minterm numbers})$   
 Example:  $F = \sum m(1, 3, 5, 7)$

#### Product of Sums (POS) form

The function is written as a product (AND) of maxterms for which the output is 0.  
 $F(A, B, C) = \prod M(\text{list of maxterm numbers})$   
 Example:  $F = \prod M(0, 2, 4, 6)$

### 7. Example (SOP form)

Given the truth table, write the function in sum of minterms.

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

**Solution:**

$$F = \sum m(1, 3, 5, 7) \\ = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + ABC$$

### 8. Example (POS form)

Write the function in product of maxterms for the same truth table.

**Solution:**

$$\text{Zeros at } m_0, m_2, m_4, m_6 \\ F = \prod M(0, 2, 4, 6) \\ = (A + B + C)(A + \bar{B} + C) \\ (\bar{A} + B + C)(\bar{A} + \bar{B} + C)$$

### 9. Key Points

- ✓ Product term = AND of literals.
- ✓ Sum term = OR of literals.
- ✓ Minterm contains all variables (n literals).
- ✓ Maxterm contains all variables (n literals).
- ✓ Any Boolean function can be written in SOP or POS form.
- ✓ For n variables, total minterms =  $2^n$  and maxterms =  $2^n$ .
- ✓ Useful in K-maps, logic minimization and digital circuit design.

### 10. Applications

- Used in simplification of Boolean functions (K-map, Quine-McCluskey).
- Used in digital logic circuit design.
- Used in designing combinational and sequential circuits.
- Helpful in computer arithmetic, control systems and VLSI design.
- Provides a standard form for analysis and implementation.

### 11. Conclusions

- Fundamental product and sum terms are the basic building blocks of Boolean expressions.
- Minterms are used for SOP form and maxterms for POS form.
- These forms help in systematic simplification and efficient circuit design.
- Understanding these concepts is essential for solving and designing logic circuits.

Learn  
Practice  
Simplify  
Succeed



# Karnaugh Maps (K-maps)

Simplify Boolean Expressions up to 2, 3, and 4 Variables

## 1. Introduction

- Karnaugh Map (K-map) is a simple and visual method to simplify Boolean expressions.
- It helps to minimize logic expressions and reduce the number of gates in digital circuits.
- It uses a two-dimensional grid where adjacent cells differ by only one bit.
- Works for 2, 3, 4 (and more) variables.
- Only for SOP (sum of products) or POS (product of sums) form.

## 2. Rules for K-map Simplification

- 1 Place 1 for minterms (and 0 for others) in the K-map.
- 2 Adjacent cells differ by only one variable (use Gray code ordering).
- 3 Group 1's in powers of 2 (1, 2, 4, 8 ...).
- 4 Groups can be horizontal, vertical, and can also wrap around.
- 5 Choose the largest possible groups.
- 6 Each 1 must be included in at least one group.
- 7 A cell can be part of more than one group.
- 8 The simplified expression is obtained by eliminating the variables that change within a group.

## 3. Gray Code Ordering

| Number of Variables | Gray Code Sequence                                                                             |
|---------------------|------------------------------------------------------------------------------------------------|
| 2                   | 00, 01, 11, 10                                                                                 |
| 3                   | 000, 001, 011, 010, 110, 111, 101, 100                                                         |
| 4                   | 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, 1101, 1111, 1110, 1010, 1011, 1001, 1000 |

## 4. 2-Variable K-map

| B | 0              | 1              |
|---|----------------|----------------|
| A |                |                |
| 0 | m <sub>0</sub> | m <sub>1</sub> |
| 1 | m <sub>2</sub> | m <sub>3</sub> |

Cell Positions:

$$\begin{aligned} m_0 &= \bar{A}\bar{B} \\ m_1 &= \bar{A}B \\ m_2 &= A\bar{B} \\ m_3 &= AB \end{aligned}$$

Adjacent cells: horizontal or vertical (also wrap around).

## 5. 2-Variable K-map Examples

Example 1:  $F(A, B) = \sum m(0, 1)$

| B | 0 | 1 |
|---|---|---|
| A |   |   |
| 0 | 1 | 1 |
| 1 | 0 | 0 |

$$F = \bar{A}$$

Example 2:  $F(A, B) = \sum m(1, 3)$

| B | 0 | 1 |
|---|---|---|
| A |   |   |
| 0 | 0 | 1 |
| 1 | 0 | 1 |

$$F = B$$

## 6. 3-Variable K-map

| BC | 00             | 01             | 11             | 10             |
|----|----------------|----------------|----------------|----------------|
| A  |                |                |                |                |
| 0  | m <sub>0</sub> | m <sub>1</sub> | m <sub>3</sub> | m <sub>2</sub> |
| 1  | m <sub>4</sub> | m <sub>5</sub> | m <sub>7</sub> | m <sub>6</sub> |

Note: Column order is in Gray code (00, 01, 11, 10).

## 7. 3-Variable Example

Simplify:  $F(A, B, C) = \sum m(0, 1, 2, 3)$

| BC | 00 | 01 | 11 | 10 |
|----|----|----|----|----|
| A  |    |    |    |    |
| 0  | 1  | 1  | 1  | 1  |
| 1  | 0  | 0  | 0  | 0  |

$$F = \bar{A}$$

## 8. 4-Variable K-map

| CD | 00              | 01              | 11              | 10              |
|----|-----------------|-----------------|-----------------|-----------------|
| AB |                 |                 |                 |                 |
| 00 | m <sub>0</sub>  | m <sub>1</sub>  | m <sub>3</sub>  | m <sub>2</sub>  |
| 01 | m <sub>4</sub>  | m <sub>5</sub>  | m <sub>7</sub>  | m <sub>6</sub>  |
| 11 | m <sub>12</sub> | m <sub>13</sub> | m <sub>15</sub> | m <sub>14</sub> |
| 10 | m <sub>8</sub>  | m <sub>9</sub>  | m <sub>11</sub> | m <sub>10</sub> |

Both rows and columns are in Gray code.

## 9. 4-Variable Example 1

Simplify:  $F(A, B, C, D) = \sum m(0, 1, 2, 3, 8, 9, 10, 11)$

| CD | 00 | 01 | 11 | 10 |
|----|----|----|----|----|
| AB |    |    |    |    |
| 00 | 1  | 1  | 1  | 1  |
| 01 | 1  | 1  | 1  | 1  |
| 11 | 0  | 0  | 0  | 0  |
| 10 | 0  | 0  | 0  | 0  |

$$F = \bar{A}$$

## 10. 4-Variable Example 2

Simplify:  $F(A, B, C, D) = \sum m(0, 2, 8, 10)$

| CD | 00 | 01 | 11 | 10 |
|----|----|----|----|----|
| AB |    |    |    |    |
| 00 | 1  | 0  | 0  | 1  |
| 01 | 0  | 0  | 0  | 0  |
| 11 | 0  | 0  | 0  | 0  |
| 10 | 1  | 0  | 0  | 1  |

$$F = \bar{B}\bar{D}$$

## 11. Grouping Patterns

| Group Size | Shape | Terms Eliminated |
|------------|-------|------------------|
| 1          |       | None             |
| 2          |       | 1 variable       |
| 4          |       | 2 variables      |
| 8          |       | 3 variables      |
| 16         |       | 4 variables      |

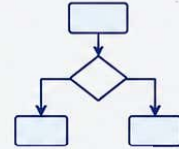
## 12. Key Takeaways

- ✓ K-maps provide a simple and visual method for minimization.
- ✓ Adjacent cells differ by only one bit (Gray code).
- ✓ Group 1's in powers of 2.
- ✓ Larger groups give simpler expressions.
- ✓ Can be used for 2, 3, 4 variables.
- ✓ Useful in digital circuit design to reduce the number of gates.

Learn  
Practice  
Simplify  
Succeed



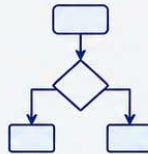
1010101010  
0101010101  
1010101010  
0101010101



# Unit - IV



10110  
01001  
10101





Process  
Control  
Repeat!

# Microprocessor

The Brain of a Computer System

Small Chip  
Big Possibilities!

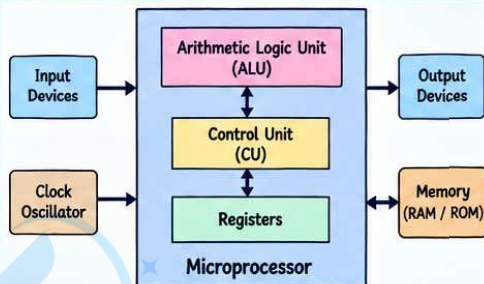
"A Microprocessor executes instructions, processes data and controls the entire system."

## 1. What is a Microprocessor?

- A microprocessor is a programmable integrated circuit (IC) that performs arithmetic, logic, control and data processing functions.
- It is the CPU (Central Processing Unit) on a single chip.
- It reads instructions from memory, processes data and gives the result.
- It is a multipurpose, general-purpose device used in computers, embedded systems, calculators, home appliances, etc.

"A microprocessor is a computer on a chip."

## 2. Block Diagram of a Microprocessor



"The microprocessor communicates with input, output and memory through address, data and control buses."

## 3. Main Components

### Arithmetic Logic Unit (ALU)

- Performs arithmetic operations (add, sub, etc.)
- Performs logical operations (AND, OR, NOT, etc.)

### Control Unit (CU)

- Controls the flow of data and instructions.
- Generates control signals.

### Registers

- Small, high-speed storage locations.
- Store data, instructions and addresses temporarily.

## 4. Buses in a Microprocessor

### Address Bus

- Carries the address of memory location or I/O device.
- Unidirectional (CPU → Memory/I/O).
- Width determines maximum addressable memory (e.g., 16 bits = 64 KB).

### Data Bus

- Carries actual data.
- Bidirectional (CPU ↔ Memory/I/O).
- Width determines data processing capacity (e.g., 8-bit, 16-bit, 32-bit).

### Control Bus

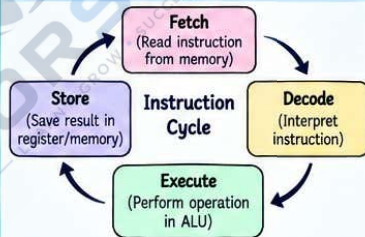
- Carries control signals (Read, Write, Interrupt, etc.).
- Both input and output signals.

## 5. Working of a Microprocessor

- Fetch** - Instruction is read from memory.
- Decode** - Instruction is decoded by control unit.
- Execute** - ALU performs the operation.
- Memory Access** - If required, data is read/written to memory.
- Write Back** - Result is stored in register or memory.

"Fetch → Decode → Execute → Repeat"

## 6. Instruction Cycle



## 7. Types of Microprocessors

| Type   | Description                                           |
|--------|-------------------------------------------------------|
| 8-bit  | Processes 8 bits of data at a time (e.g., 8085, 8088) |
| 16-bit | Processes 16 bits of data (e.g., 8086, 80286)         |
| 32-bit | Processes 32 bits of data (e.g., 80386, Pentium)      |
| 64-bit | Processes 64 bits of data (e.g., Core i3, i5, i7, i9) |

## 9. Examples of Microprocessors



## 10. Advantages

- ✓ Small size and low cost
- ✓ High processing speed
- ✓ Reliable and durable
- ✓ Low power consumption
- ✓ Multipurpose and versatile
- ✓ Widely used in many applications

## 11. Limitations

- ✗ Limited processing power (older versions)
- ✗ Requires external memory and I/O support
- ✗ Not suitable for very complex tasks (older versions)
- ✗ Performance depends on clock speed
- ✗ Needs proper cooling in high-speed processors

## 12. Key Takeaways

- ✓ Microprocessor is the CPU on a single chip.
- ✓ It consists of ALU, CU and Registers.
- ✓ It uses address, data and control buses.
- ✓ It works in a fetch-decode-execute cycle.
- ✓ Types: 8-bit, 16-bit, 32-bit, 64-bit.
- ✓ Used in a wide range of applications.
- ✓ It revolutionized the digital world.

"Smaller Chips... Bigger Futures!"



- 8. Applications**
- Personal Computers
  - Mobile Devices
  - Calculators
  - Automotive Systems
  - Industrial Automation
  - Home Appliances
  - Embedded Systems
  - Robotics
  - Medical Equipment
  - Networking Devices

From  
PC to Phone  
everywhere  
Microprocessors  
are there! 😊



Small Blocks  
Big Possibilities! 😊

# Block Diagram of a Microcomputer

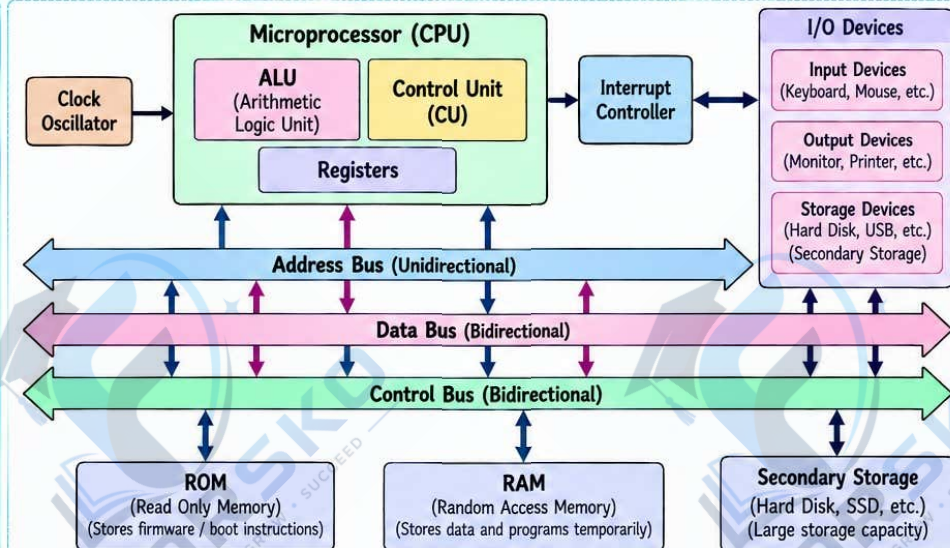
## Main Components and Their Interconnection

Microcomputer  
Brings Ideas to Life! 😊

### 1. Introduction

- A microcomputer is a complete computer system built around a **microprocessor**.
- It consists of CPU, memory, I/O devices and buses.
- All the components are connected through **address bus**, **data bus** and **control bus**.
- It is used in personal computers, embedded systems, and various digital devices.

"A microcomputer is a computer on a single chip based system."



### 2. Main Components

#### 1 Microprocessor (CPU)

- Performs arithmetic, logic, control and data processing operations.
- Consists of ALU, Control Unit and Registers.

#### 2 Memory

- ROM** : Stores permanent instructions (firmware/BIOS).
- RAM** : Stores data and programs temporarily.

#### 3 I/O Devices

- Input : Keyboard, Mouse, etc.
- Output : Monitor, Printer, etc.
- Storage : Hard Disk, USB, etc.

#### 4 Buses

- Address Bus** : Carries address (unidirectional).
- Data Bus** : Carries data (bidirectional).
- Control Bus** : Carries control signals (read, write, interrupt, etc.).

### 3. Function of Each Bus

#### Address Bus

- Carries the address of memory location or I/O port.
- Unidirectional (CPU → Memory/I/O).
- Width determines maximum addressable memory (e.g., 16 bits = 64 KB, 32 bits = 4 GB).

#### Control Bus

- Carries control signals like Read, Write, Interrupt, Reset, etc.
- Both input and output signals.
- Synchronizes all operations.

#### Data Bus

- Carries actual data.
- Bidirectional (CPU ↔ Memory/I/O).
- Width determines data processing capacity (e.g., 8-bit, 16-bit, 32-bit).

### 4. Working of a Microcomputer

- CPU places the address on the address bus.
- Control signals are sent through the control bus (e.g., Read/Write).
- Data is transferred through the data bus.
- Data is stored in memory or sent to I/O devices.
- I/O devices communicate via the I/O interface and interrupt controller.

All components work together through buses to perform the desired operation.

### 5. Applications

- Personal Computers (Desktop/Laptop)
- Office Automation
- Educational Systems
- Industrial Control Systems
- Embedded Systems
- Home Appliances (Microwave, Washing Machine, etc.)
- Medical Instruments
- Communication Devices
- Automotive Systems

### 6. Key Takeaways

- ✓ A microcomputer integrates CPU, memory and I/O on a single system.
- ✓ Buses provide the interconnection between components.
- ✓ Address bus is unidirectional, data bus is bidirectional, control bus carries control signals.
- ✓ ROM stores permanent data, RAM stores temporary data.
- ✓ Used in a wide range of applications in daily life.

Same Concept, Infinite Possibilities! 😊



Better  
Processor  
Better Performance ! 😊

# Evaluation of Microprocessor

How to Compare and Choose the Right Microprocessor ?

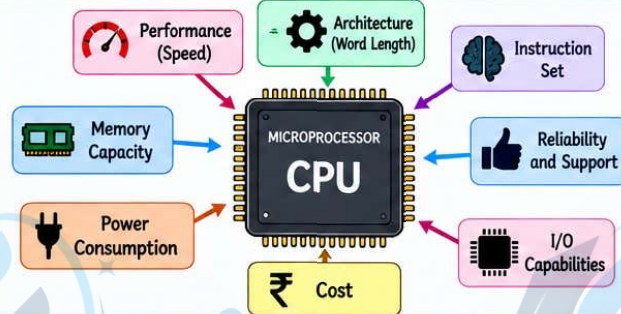
Different Processors  
Different Strengths  
Choose Smart ! 😊

## 1. What is Evaluation of Microprocessor ?

- Evaluation of a microprocessor means analyzing and comparing its features, performance and capabilities to decide its suitability for a particular application.
- It helps in selecting the best processor based on requirements like speed, cost, power, and application.

"Evaluation helps to find the right processor for the right task."

## 2. Key Parameters for Evaluation



## 3. Important Evaluation Criteria

| Criteria                   | Description                                                                   |
|----------------------------|-------------------------------------------------------------------------------|
| Clock Speed                | Higher clock speed means faster execution (e.g., 1 GHz, 3.5 GHz).             |
| Word Length                | 8-bit, 16-bit, 32-bit, 64-bit (more bits → better processing capability).     |
| Instruction Set            | Number and type of instructions supported.                                    |
| Memory Addressing Capacity | Amount of memory that can be addressed (e.g., 16-bit = 64 KB, 32-bit = 4 GB). |
| Data Transfer Rate         | Speed of data transfer between processor, memory and I/O devices.             |
| Power Consumption          | Lower power is better (important for portable devices).                       |
| Cost                       | Should be affordable and cost-effective.                                      |
| Reliability & Support      | Availability, stability, and manufacturer support.                            |

## 4. Performance Evaluation

- Measured in terms of:

- ✓ Clock speed (Hz)
- ✓ Instructions per second (IPS)
- ✓ MIPS (Million Instructions Per Second)
- ✓ Execution time
- ✓ Throughput



More MIPS = Higher Performance  
(But not the only factor !)

## 5. Types of Evaluation

| Type                         | Focus Area                                                                      |
|------------------------------|---------------------------------------------------------------------------------|
| Performance Evaluation       | Speed, MIPS, execution time, throughput                                         |
| Functional Evaluation        | Correct execution of instruction set                                            |
| Power Evaluation             | Power consumption and heat dissipation                                          |
| Cost Evaluation              | Price vs features                                                               |
| Application-based Evaluation | Suitability for specific use (e.g., embedded systems, personal computers, etc.) |

## 6. Example: Comparison of Microprocessors

| Feature           | 8085 (8-bit)  | 8086 (16-bit) | 80386 (32-bit)  | Pentium (32-bit) | Core i5 (64-bit) |
|-------------------|---------------|---------------|-----------------|------------------|------------------|
| Clock Speed       | 3 MHz         | 5–10 MHz      | 16–33 MHz       | 60–200 MHz       | 2.4–4.2 GHz      |
| Word Length       | 8-bit         | 16-bit        | 32-bit          | 32-bit           | 64-bit           |
| Memory Addressing | 64 KB         | 1 MB          | 4 GB            | 4 GB             | 16 GB+           |
| Instruction Set   | ~74           | ~117          | ~200+           | Enhanced         | Advanced         |
| Power Consumption | Low           | Moderate      | Moderate        | Higher           | Optimized (Low)  |
| Typical Use       | Early systems | PC (early)    | Multitasking OS | High performance | Modern computing |

## 7. Factors Affecting Evaluation

- ✓ Technology (nm fabrication)
- ✓ Cache memory size
- ✓ Multi-core support
- ✓ Compatibility with software/hardware
- ✓ Heat dissipation and power management
- ✓ Price and market availability



Newer technology  
= Higher performance  
+ Lower power

## 8. Applications of Evaluation

- Choosing a processor for a PC or Laptop
- Designing embedded systems
- Selecting processors for industrial use
- Comparing processors in smartphones
- Research and development
- Finding best value for money

## 9. Conclusion

- Evaluation of a microprocessor is essential before selecting it for any application.
- It helps to balance performance, cost, power and features.
- The best processor is the one that meets the application requirements efficiently.

"Evaluate → Compare → Choose → Perform Better"

## 10. Key Takeaways

- ✓ Higher clock speed improves performance.
- ✓ Memory and I/O capability are equally important.
- ✓ Power and cost matter for real-world use.
- ✓ Evaluation depends on application needs.
- ✓ Newer processors provide better performance with lower power consumption.
- ✓ Always compare features before choosing.

Right Processor = Smarter Solution ! 😊



Microprocessors and Interfacing

"Evaluate Today, Innovate Tomorrow !" 😊

Learn | Compare | Apply | Grow

Understand  
Architecture  
Build the Logic !



# Architecture Of 8085 Microprocessor

Simple Structure ... Powerful Capabilities !



8085  
Small Chip  
Big Impact !



## 1. Introduction

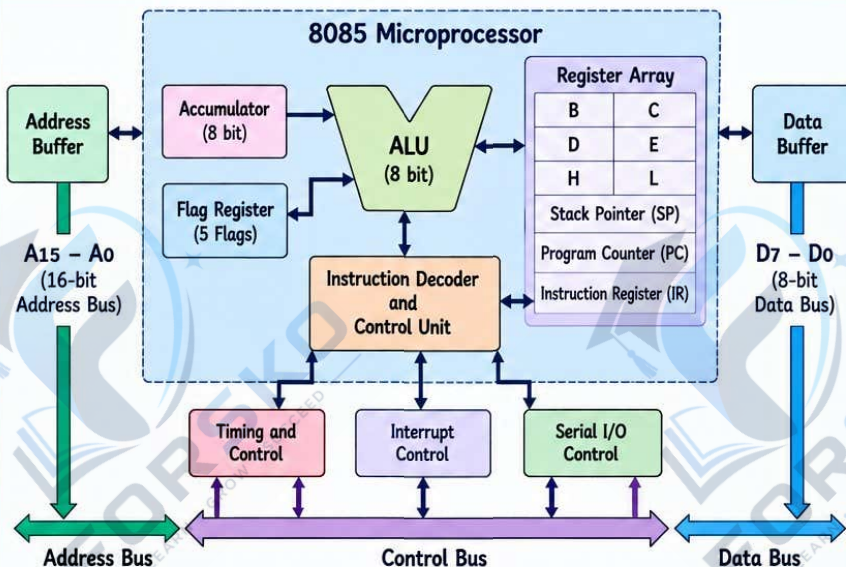
- 8085 is an 8-bit microprocessor introduced by Intel in 1976.
- It is a general-purpose microprocessor.
- It can process 8-bit data and address 16-bit memory locations (64 KB).
- Its architecture consists of ALU, Registers, Control Unit, Instruction Register, Instruction Decoder, and buses.

"8085 – A simple yet powerful 8-bit microprocessor that changed the world of computing."

## 2. Key Features

- ✓ 8-bit data bus (processes 8 bits at a time)
- ✓ 16-bit address bus (can address 64 KB memory)
- ✓ 74 instructions
- ✓ 5 interrupt lines
- ✓ Serial I/O (SID, SOD)
- ✓ Operates on +5V single power supply
- ✓ Widely used in educational and industrial applications

## 3. Block Diagram of 8085 Architecture



## 4. Arithmetic Logic Unit (ALU)

- Performs arithmetic operations (add, sub, increment, decrement).
- Performs logical operations (AND, OR, XOR, Compare).
- Operates on 8-bit data.

## 5. Registers

- Accumulator (A) – 8-bit, stores result of ALU operations.
- General Purpose Registers – B, C, D, E, H, L (8-bit each).
- Program Counter (PC) – 16-bit, holds address of next instruction.
- Stack Pointer (SP) – 16-bit, points to top of stack.
- Instruction Register (IR) – 8-bit, holds current instruction.
- Flag Register – 5 flags (S, Z, AC, P, CY).

## 6. Flag Register

| D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|----|----|----|----|----|----|----|----|
| S  | Z  | -  | AC | -  | P  | -  | CY |

S – Sign      Z – Zero      AC – Auxiliary Carry  
P – Parity      CY – Carry

## 7. Control Unit

- Decodes the instruction from IR.
- Generates control and timing signals.
- Controls the flow of data between CPU, memory and I/O devices.
- Manages all operations inside the processor.

## 8. Buses

- Address Bus (16-bit)** – Carries address from CPU to memory/I/O (unidirectional).
- Data Bus (8-bit)** – Carries data between CPU, memory and I/O (bidirectional).
- Control Bus** – Carries control and timing signals (e.g., Read, Write, IO/M, ALE, etc.).

## 9. Interrupt and Serial I/O

- Interrupts:** TRAP, RST 7.5, RST 6.5, RST 5.5, INTR (maskable and non-maskable).
- Serial I/O:** SID (Serial Input Data), SOD (Serial Output Data).
- Provides synchronization with external devices.

## 10. Applications

- ✓ Educational purposes
- ✓ Embedded systems
- ✓ Industrial automation
- ✓ Process control
- ✓ Data acquisition systems
- ✓ Legacy systems and hardware interfacing

"Small Architecture ... Endless Possibilities !"



Learn Implement Innovate Grow



8085 Microprocessor

The Foundation of Modern Computing

Learn  
Understand  
Remember ! 

# 8085 Microprocessor Pin Diagram



Small Chip  
Big Capabilities ! 

40 Pins ... Many Functions ... One Powerful Processor !

## 1. Introduction

- 8085 is an 8-bit microprocessor introduced by Intel in 1976.
- It has 40 pins.
- These pins are used for address, data, control, power and other functions.
- Understanding the pin diagram helps in interfacing 8085 with memory and I/O devices.

"Pins connect the 8085 to the outside world."

## 2. Pin Classification

The 40 pins of 8085 can be grouped into:

- Address bus pins (16)
- Data bus pins (8)
- Control and status pins (10)
- Power supply pins (2)
- Clock pins (3)
- Reset pins (2)
- Serial I/O pins (2)
- Other pins (2)

"Different pins, different roles, but all work together !"

## 3. 8085 Pin Diagram



## 5. Key Takeaways

- 8085 has 40 pins.
- Pins are grouped based on their functions.
- Address bus = 16 pins (A0 - A15).
- Data bus = 8 pins (AD0 - AD7).
- Control, status, clock, reset, interrupt and serial I/O pins help in communication with external devices.
- Vcc and Vss provide power supply.

"Understand the Pins  
Build the Possibilities !" 

## 4. 8. Functions of Each Pin Group

### 1 Clock Pins (1, 2, 37)

- X1, X2 : Crystal input pins (connects a crystal oscillator).
- CLK OUT : Provides system clock to external devices.

### 2 Reset Pins (3)

- RESET OUT : Indicates that the processor has been reset.

### 7 Serial I/O Pins (4, 5)

- SOD : Serial Output Data (1-bit).
- SID : Serial Input Data (1-bit).

### 5 Address Bus Pins (21 - 30)

- A8 - A15 : Higher-order 8 bits of address bus.
- These pins provide 16-bit addressing (64 KB memory space).

### 6 Power Supply Pins (20, 40)

- Vss : Ground (0V)
- Vcc : +5V DC power supply.

### 3 Control and Status Pins

- IO/M : Selects I/O or Memory operation.
- S1 : Status output (used with IO/M).
- RD : Read control signal.
- WR : Write control signal.
- ALE : Address Latch Enable (to separate address and data).
- READY : Indicates device is ready.
- HOLD : Request for bus access.
- HLDA : Hold acknowledge.
- INTA : Interrupt acknowledge.

### 4 Data Bus Pins (12 - 19)

- AD0 - AD7 : Lower 8 bits are data bus (D0 - D7).
- These pins are multiplexed with lower address bus (A0 - A7).
- Used to transfer 8-bit data.

### 8 Interrupt Pins (6 - 10)

- TRAP : Non-maskable, highest priority.
- RST 7.5, RST 6.5, RST 5.5 : Maskable interrupts (vectored).
- INTR : General interrupt request.

## Applications

- Interfacing memory (RAM, ROM)
- Interfacing I/O devices (keyboard, display, sensors)
- Used in embedded systems and industrial control
- Educational and research purposes
- Foundation for modern microprocessors



From Basics  
to Brilliant  
Ideas !